

Multi-Modal Deep Transfer Learning for Business Signals Detection in Social Media Communications

Dominika Agnieszka Długosz

A dissertation submitted in partial fulfilment
of the requirements for the degree of
Master of Science
of the
University of Aberdeen.



The School of Natural and Computing Sciences
Department of Computing Science

2020

Declaration

No portion of the work contained in this document has been submitted in support of an application for a degree or qualification of this or any other university or other institution of learning. All verbatim extracts have been distinguished by quotation marks, and all sources of information have been specifically acknowledged.

Signed:

Date: August 14, 2020

Abstract

A business signal can be defined as information of potential relevance to sales people or businesses more generally. It can be related to business opportunities, trends in the market, or marketing content. This dissertation describes the development of a deep transfer learning model for the recognition of business signals on the Instagram social media platform.

Owing to the multi-modal character of the data, two independent pretrained models were used: RoBERTa for the processing of textual data (Instagram post descriptions) and Xception for images. Additional model structures were built for the processing of Instagram-specific tags and hashtags. The complete developed deep learning model had a multiple-input, single-output structure. The different inputs were first processed by dedicated model branches, after which the outputs were combined and subject to further, joint analysis, to yield a single prediction, marking a post as either relevant or irrelevant from a business perspective.

The model was trained on approximately 2000 post examples retrieved from almost a hundred Instagram profiles. 250 more samples - a validation dataset - were used to control the learning process. The training was executed in several stages, where the pretrained submodels were gradually *unfrozen* to allow their controlled fine-tuning.

The model's performance was assessed against a separate test set of 250 samples. 92% accuracy was achieved, with precision and recall of 84% and 90% respectively. Of the mistakes made by the model on the test set, the majority were insignificant (the posts were mostly weak business signals).

The project was realised in cooperation with DeepMiner - an Aberdonian software company building an AI-driven business news search engine. The developed and trained model will be incorporated in DeepMiner's product, extending their data sources - news services and Twitter - to the Instagram platform.

Acknowledgements

I would like to express my gratitude to several people who supported and guided me throughout this project. First of all, I would like to thank Áine Uí Ghiollágain, the CTO of DeepMiner, who welcomed me warmly at the company and provided me with invaluable advice - in particular during the strenuous final days. I would also like to acknowledge my academic supervisor, Dr Georgios Leontidis, for many useful, practical insights and suggestions. I could not forget my university friends, who made the last year an inspiring adventure, filled with shared efforts and laugh. Last but not least, I would like to thank my fiancé, Kamil, for the inexhaustible reserves of love, patience, and chocolate.

Contents

1	Introduction	7
1.1	Objectives	10
1.2	Outline	10
2	Background	11
2.1	Neural networks	11
2.1.1	Basic concepts	11
2.1.2	Neural network layer types	11
2.1.3	Convolutional neural networks	12
2.1.4	Recurrent neural networks	13
2.1.5	Neural network regularisation strategies	13
2.2	Transfer learning	14
2.2.1	Definitions	14
2.2.2	Comparison between transfer learning and traditional machine learning .	15
2.2.3	Classification of transfer learning methods	16
2.2.4	Deep transfer learning	16
3	Literature Review	17
3.1	Deep learning for computer vision	17
3.1.1	Deep learning architectures for image processing	17
3.1.2	Computer vision with deep transfer learning	17
3.2	Deep learning for natural language processing	19
3.2.1	NLP with deep transfer learning	19
3.2.2	Datasets for NLP with transfer learning	20
3.3	Deep learning for hybrid tasks	20
3.3.1	Multi-modal deep learning for social media data	21
3.3.2	Image-to-text generation	22
4	Methods	23
4.1	Data	23
4.1.1	Data acquisition	23
4.1.2	Description of data collected	25
4.2	Data processing pipeline	27
4.2.1	Preprocessing and feature extraction	27

4.2.2	Model definition	32
4.2.3	Model training	34
4.2.4	Evaluation methods	36
4.2.5	Storing the trained model	37
5	Results	38
5.1	Training process	38
5.2	Trained model performance	39
5.3	Qualitative assessment of the model performance	40
6	Conclusions	44
6.1	Limitations	45
6.2	Future work	45
A	Model Architecture	52
B	Instagram Scraper - Google Colab Setup	56
B.1	Instagram Scraper Manual	57
B.2	Instagram Scraper - Google Colab Setup	61
C	The Dataset	64
D	Results	67
E	Instarget User Manual	71
E.1	Installation and usage	71
E.1.1	Local installation	71
E.1.2	Using Instarget on Google Colab	72

Chapter 1

Introduction

Knowledge is power, says a well-known Latin aphorism. It could not be more true in the context of entrepreneurship in the capitalist world. The ability to keep an ear to the ground and an eye out for smoke signals could decide the fate of a company of a venturesome individual. To be, or not to be, that is the question of information. In today's world barrage of the latter, finding the right clues could be a needle-in-a-haystack type of activity, consuming large amounts of the precious time, so frequently equated with money.

Sifting through the noise is the motto of DeepMiner, an Aberdeen-based software company. Their product - an AI-driven B2B search engine - provides their customers with up-to-the-minute business-relevant information, retrieved from major news portals and Twitter. In contrast to the conventional list view, the results are linked to people, organisations, and geographical locations, and displayed in a form of a dynamic spidergram, as shown in Fig. 1.1 and 1.2, or plotted on a world map (Fig. 1.3). These and further functionalities of the search engine can be explored at www.deepminer.co.uk.

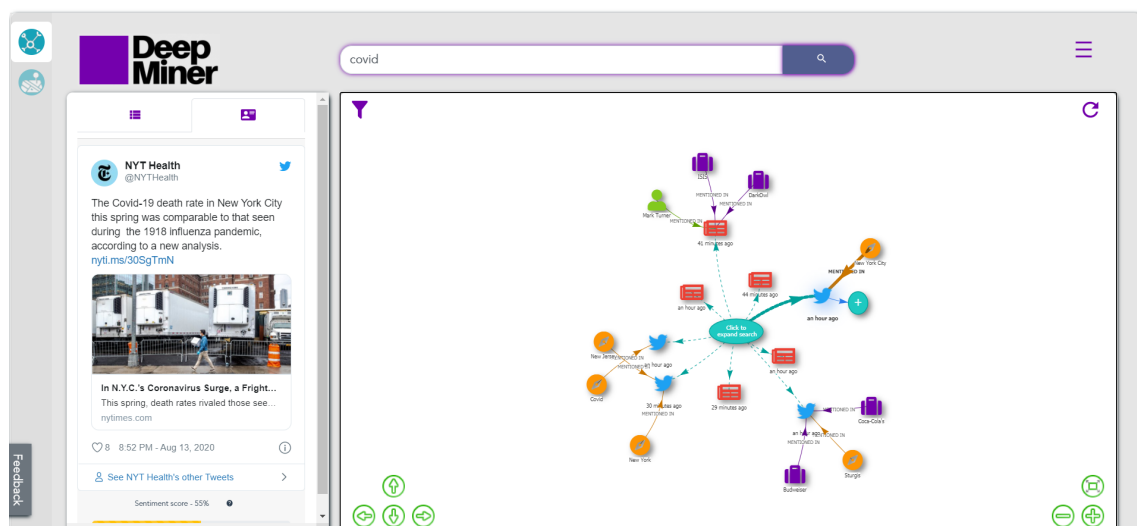


Figure 1.1: DeepMiner dashboard

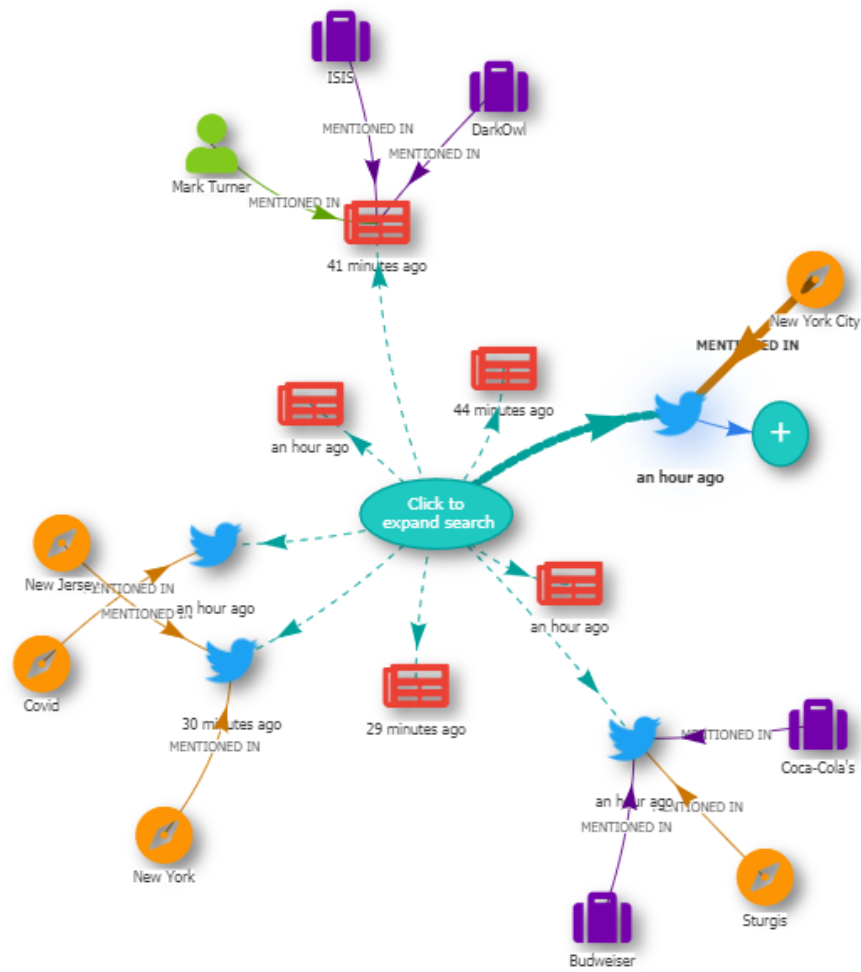


Figure 1.2: DeepMiner search result graph for query ‘coronavirus’

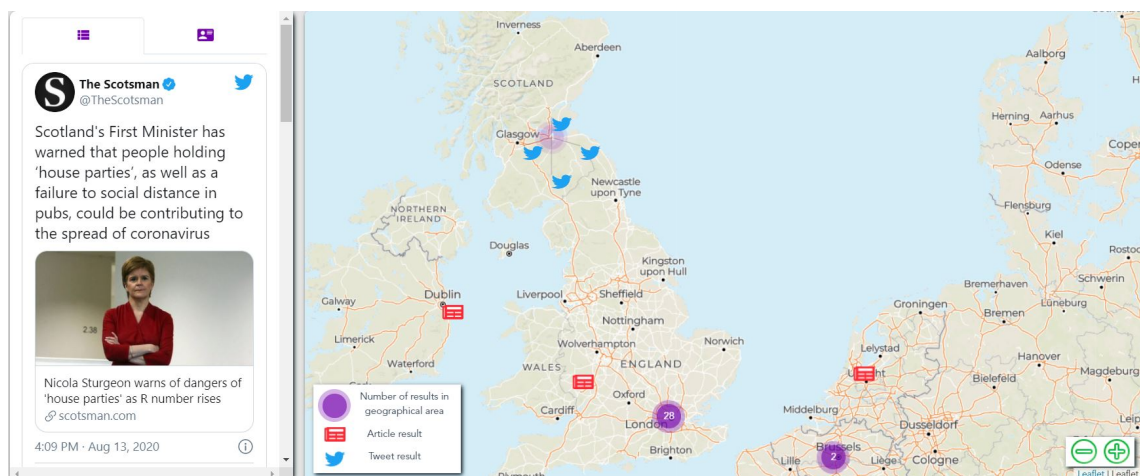


Figure 1.3: ‘Coronavirus’ query search results in DeepMiner in different geographical locations

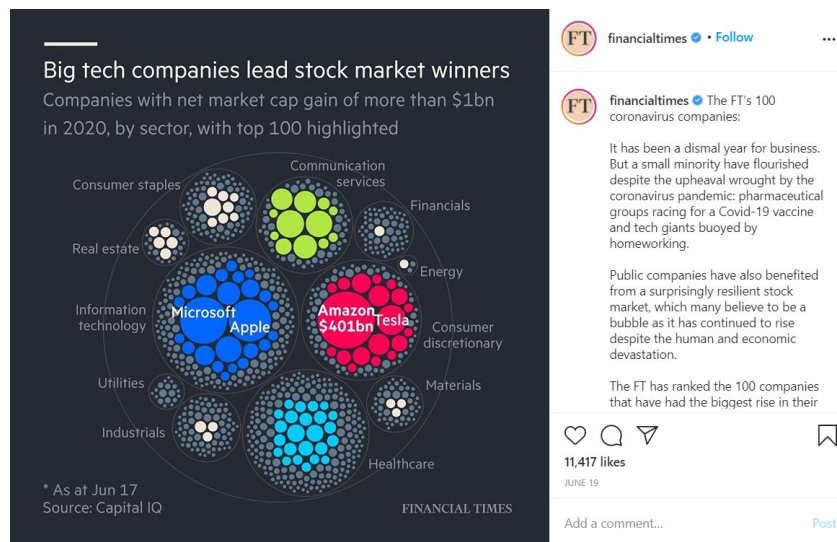


Figure 1.4: Business-relevant Instagram post example



Figure 1.5: Example of an Instagram post irrelevant for business users

The company is currently looking to extend its set of information sources by incorporating the outputs of other social media platforms, such as Instagram. Initially meant for graphical and video documentation of mundane everyday activities, Instagram gradually attracted the attention of professional users. This includes different types of companies, government organisations, and officials, as well as news portals of general and more specialised profiles. However, even business-oriented Instagram users with verified professional accounts post diverse content with various extent of business relevance, as shown in Fig. 1.4 and 1.5.

The communication of information in Twitter and news feeds occurs primarily by textual means, sometimes accompanied by images to enrich the user experience. In contrast to that, in Instagram the focus has inherently been put on visuals - images or videos; the associated text could often be treated as a complementary, but secondary information source. Therefore, the effective filtering of noise requires joint analysis of the two forms of data.

1.1 Objectives

The work described in this thesis was focused on analysis of data retrieved from Instagram to identify *business signals* (or *business search targets*) - relevant information of potential business value. As defined by the CEO of DeepMiner, business signals can generally fall into three main categories. The first is related to business opportunities. This includes information about potential funding, investments, concluded or broken business partnerships, new products emerging on the market etc. The second type are trends in the industry, including already observed and predictable changes. The final category of business signals involves information which can be used to generate marketing content and help companies achieve and sustain their popularity on the market.

There were two main objectives for the project. First of all, the feasibility of the identification of business signals in Instagram data should be assessed. The definition of business signals might be simple; however, in the real world, the border between relevant and irrelevant information often renders fuzzy. The second objective was to develop a deployable machine learning model for the realisation of the classification task. If successful, the model should later be incorporated into the DeepMiner search engine.

1.2 Outline

This dissertation is organised into 6 chapters, as described below.

- Chapter 1: **Introduction** - a presentation to the topic area and the problem statement
- Chapter 2: **Background** - terminology and concepts related to the technologies used in this project, including deep learning and transfer learning
- Chapter 3: **Literature review** - a comparison of state-of-the-art solutions in the field of natural language processing and computer vision, as well as a summary of different projects adopting these technologies to practical problems
- Chapter 4: **Methods** - a description of what, how and why work has been carried out within this project, including the collection and preprocessing of the data, the design and training of a deep-learning model developed for the identification of business signals, and applied evaluation methods
- Chapter 5: **Results** - presentation of the achieved results with quantitative and qualitative analysis
- Chapter 6: **Conclusions** - a summary and critical analysis of the realised work, as well as possible future development directions.

Chapter 2

Background

2.1 Neural networks

2.1.1 Basic concepts

The term Artificial Neural Networks (ANNs or simply NNs) relates to one of several families of machine learning (ML) algorithms inspired by biological structures and processes - in this case, the neural networks of animal brains. NNs are comprised of artificial neurons, frequently organised in layers. The first and last layers of a multi-layer network are called *input* and *output* layers respectively; layers in between them are referred to as *hidden* [1, 2].

Inter-neuron connections (edges), associated with weights, allow for the transmission of numerical information (signal) through the network. Upon receiving a set of signals, a neuron computes a weighted sum of the inputs and can apply a non-linear operation on the result, which is then passed to subsequent connected neurons [1, 2].

The training of a neural network consists of adjusting the weights on the edges. For each example (or a *batch* of examples) from the training dataset, the network computes a prediction, compares it to the ground truth, and (if necessary) applies a correction to the edge weights, working backwards from the output towards the input layer. This is referred to as *backpropagation* algorithm. A single passage of the entire training dataset through the network is referred to as a training *epoch*. Multiple epochs are required to train a neural network, with the exact number being dependent on the size and properties of both the data and the model. The total prediction error, called the *loss*, is expected to decrease throughout the training process until reaching convergence (when assessed on the training dataset) [1, 2].

2.1.2 Neural network layer types

There are multiple types of layer designs for artificial neural networks. This section briefly describes some of the most widely used layer types and their main use cases.

Fully connected or *dense layers* are one of the fundamental layer types. As the name suggests, every neuron in such a layer is connected to every neuron in the previous layer. The design implies a combinatorial explosion of the number of connections with the increase in size of the involved layers, therefore rendering dense layers computationally expensive and poorly scalable. However, dense layers are still the design of choice for certain applications, including output layers (producing

the final classification or regression result). Dense layers are the core of Multilayer Perceptron (MLP) networks, frequently referred to as *vanilla* neural networks [2].

Convolution (or *convolutional*) *layer* is based on the fully connected layer structure. It is designed to apply a set of convolutional filters (kernels) to single- or multi-dimensional inputs. The weights constituting the filters, and hence the features extracted from the inputs, are not specified up front, but learned as part of the training process. Convolutional layers form the core of Convolutional Neural Networks, as described in section 2.1.3.

Recurrent layers refer to layers with feedback connections, i.e. (some of) the layer outputs are connected to its inputs (with a set delay). Owing to those connections, a recurrent layer maintains an internal state useful for processing sequential data. Layers without feedback connections are referred to as *feedforward* [1].

Embedding layers are primarily used for the processing of text. Tokens extracted from text are usually encoded with integers or (equivalently) in one-hot encoding form, producing high-dimensional, sparse vectors. Embedding is a technique of transforming those vectors into dense vectors in lower-dimensional space. The goal of embedding is also to produce a representation which captures semantic similarities between words, i.e. words describing related concepts should be represented as approximately parallel vectors (yielding high cosine similarity score). An embedding layer, positioned at the beginning of a text processing network, is designed to learn meaningful embeddings for words in the encoded text [3].

Pooling layers, sometimes referred to as *subsampling layers*, are designed to reduce the resolution of the data (*feature maps*) by downsampling. In principle, a pooling layer divides the input feature map into non-overlapping patches usually (e.g. 2x2 pixel patches for 2-dimensional data) and reduces each patch to a single value. There are two main pooling types: *max* (*maximum*) pooling, which picks the maximum value for each patch, and *average pooling*, calculating the mean of the patch values. In a special case, the operation can be applied to the entire feature map rather than its constituent parts; it is then referred to as *global pooling*. Pooling layers are an essential part of Convolutional Neural Networks, where they are responsible for ‘summarising’ the feature maps and help the model achieve invariance to small translations [2, 4].

2.1.3 Convolutional neural networks

The analysis of 2-dimensional data, such as images, is frequently performed with the use of Convolutional Neural Networks (CNNs). A CNN can be defined as

‘a multilayer perceptron designed specifically to recognize two-dimensional shapes with a high degree of invariance to translation, scaling, skewing, and other forms of distortion’. [2]

As the name suggests, CNNs use convolution layers as their core building blocks. In its basic version, the hidden part of a CNN architecture consists of alternating convolution and subsampling (pooling) layers. Convolution layers, consisting of multiple *feature maps*, extract local features from the input through the small *receptive fields* of each neuron. Pooling layers are then used to reduce the resolution of the feature maps [2].

CNNs utilise a mechanism referred to as *weight sharing*, by which a constraint is imposed on all neurons in a feature map to use the same set of inbound weights; in other words, each convolution filter has the same values across the input. This substantially reduces the number of free parameters in the network, making the learning process more efficient and parallelisable. It also leads to improved generalisation capabilities of the network [2].

2.1.4 Recurrent neural networks

Recurrent Neural Networks (RNNs) are a type of NN engineered for processing sequential data, e.g. one-dimensional numerical signals or text. Their characteristic feature is the presence of recurrent connections - either in a form of recurrent layers (feeding their outputs to their own inputs) or in between two different units. RNNs can produce an output for each time step in the analysed input sequence or a single output for the entire input. [4].

Feedback connections allow RNNs to exploit the causal links in sequential data, as the response of the network at a given time step depends on the previously processed information. A variation of this architecture, bidirectional RNNs, enables the analysis of both *sides* of a sample in the sequence to produce a prediction. This type of networks is useful in Natural Language Processing tasks, where context from both sides is important [4].

One of the issues associated with RNNs is the *vanishing gradient problem*. An unfolded RNN is effectively a deep network; a new layer is created at each time step. The vanishing gradient problem is a phenomenon associated with deep NNs by which gradients for layers situated closer to the input tend to decrease in an exponential fashion. Their weight updates are smaller and the learning process is therefore less efficient, with the possibility of stagnation [4].

With respect to RNNs, the problem can be circumvented by introducing *gates*. A widely used example of gated RNNs is *Long Short Term Memory* (LSTM) architecture. An LSTM network is composed of *cells*, each having input, output, and forget gates. The gates control the flow of information within and between cells, allowing some information to be maintained in the network across a number of time steps, while other irrelevant information can be discarded [4].

2.1.5 Neural network regularisation strategies

One of the main issues with training neural networks is the possibility of overfitting - a situation in which further training leads to an improvement in the performance of the model on the training dataset and a simultaneous decline with respect to the test/validation data. Overfitting can be viewed as memorising the training data rather than learning the general underlying rules to perform the operation on data the model has not trained on. It is particularly an issue in cases where the network is large and the dataset relatively small, which leads to the model having more degrees of freedom than is present in the data [5]. In this section, several regularisation methods, designed to prevent overfitting and improve model generalisation capabilities, are described.

One of the basic regularisation strategies is *early stopping*. The data is split into training and validation sets, the latter being excluded from training. The performance of the model on the validation set is monitored throughout the training process. Once it starts to decline, further training is aborted. The decline should however be considered across several epochs, so that natural fluctuations in performance do not prematurely terminate the training process [4, 5].

One of the possible scenarios leading to overfitting is when meaningful representations are learned by single neurons. As a result, the network becomes strongly dependent on a few units rather than the entire interconnected structure. This is referred to as *co-adaptation*. This phenomenon might be avoided by applying *dropout* - randomly ‘switching off’ nodes in dense layers, or ‘dropping’ connections between neurons. In practice, this is done by setting the activation function, and effectively the output of the neuron, to 0. The neurons to be dropped are chosen at random with a set *dropout rate* in each gradient descent step [4, 5, 6, 7]. Following the original paper introducing the technique, commonly applied dropout rates are 0.5 and 0.2 for hidden and visible layers respectively [6]; however, different values of dropout rates can be applied [7].

Another regularisation method is *weight regularisation*. In this approach, the loss function of the model is modified by adding a scaled weight penalty term. There are multiple types of possible penalties. Frequent choices are L1 and L2 penalties, corresponding to L1 and L2 norms of the weights vectors [4, 5]. L1-type regularisation promotes weights close to 0, resulting in a more sparse model. L2 weight regularisation, a.k.a. weight decay, makes the weights smaller without causing the sparsity effect [2].

Normalisation is a group of techniques contributing to regularisation and accelerating the training process. One of the issues associated with deep architectures is that when one layer is updated, the others are assumed to be preserved; however, in each training step, the parameters of every layer (starting from the output) are modified. Normalisation addresses this problem by *reparametrising* the network. *Batch normalisation*, the first and most popular normalisation approach, transforms the activations of a layer to zero mean and unit standard deviation for each training batch (mini-batch). It has been shown that with batch normalisation, models (CNNs in particular) require fewer training epochs and show improvements in generalisation capabilities [4, 8]. Related normalisation techniques include *layer* [9], *instance* [10], and *group* [11] *normalisation*.

2.2 Transfer learning

2.2.1 Definitions

The famous definition of the process of machine learning by Mitchell states:

‘A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P if its performance at tasks in T , as measured by P , improves with experience E ’. [12, 13]

By analogy, Thrun & Pratt define the concept of *transfer learning* (*learning to learn*) as a process by which:

‘[an algorithm’s] performance improves with experience *and* with the number of tasks’ [13]

where the tasks are assumed to belong to a *family* (i.e. they share some characteristics) [13]. The efficiency of the learning process itself in transfer learning is expected to improve with each new (albeit related) task the algorithm is presented with (although a performance decline may occur in case of *negative transfer* [14]).

Formally, a machine learning problem can be defined within a *domain* $\mathcal{D}$:

$$\mathcal{D} = \{\mathcal{X}, P(X)\} \quad (2.1)$$

where $\mathcal{X}$ - a feature space and $P(X)$ - a marginal probability distribution for $X = \{x_1, \dots, x_n\} \in \mathcal{X}$ [14, 15]. Given a domain, the definition of a *task* $\mathcal{T}$ can be given as follows:

$$\mathcal{T} = \{\mathcal{Y}, f(\cdot)\} = \{\mathcal{Y}, P(Y|X)\} \quad (2.2)$$

with $\mathcal{Y}$ denoting a label space; $f(\cdot)$ - a predictive function to be learned, mapping a feature vector $x_i \in X$ on label $y_i \in \mathcal{Y}$; $P(X|Y)$ - conditional probability distribution.

With the above notations, a formal definition of transfer learning can be stated as follows:

‘Given a source domain $\mathcal{D}_S$ with a corresponding source task $\mathcal{T}_S$ and a target domain $\mathcal{D}_T$ with a corresponding task $\mathcal{T}_T$, transfer learning is the process of improving the target predictive function $f_T(\cdot)$ by using the related information from $\mathcal{D}_S$ and $\mathcal{T}_S$, where $\mathcal{D}_S \neq \mathcal{D}_T$ or $\mathcal{T}_S \neq \mathcal{T}_T$. [14]

Following the above definition, four main transfer learning settings can be distinguished [14, 15]:

- with respect to different source and target domains, $\mathcal{D}_S \neq \mathcal{D}_T$:
 1. Differing feature spaces, $\mathcal{X}_S \neq \mathcal{X}_T$ - referred to as *heterogeneous* transfer learning (as opposed to *homogeneous* transfer learning, $\mathcal{X}_S = \mathcal{X}_T$).
 2. Differing marginal probability distributions, $P_S \neq P_T$, a.k.a. *frequency feature bias*.
- with respect to different source and target tasks, $\mathcal{T}_S \neq \mathcal{T}_T$:
 3. Differing label spaces, $\mathcal{Y}_S \neq \mathcal{Y}_T$ (e.g. binary to multi-class classification transfer).
 4. Differing conditional probability distributions, $P(Y_S|X_S) \neq P(Y_T|X_T)$, a.k.a. *context feature bias*.

2.2.2 Comparison between transfer learning and traditional machine learning

The expected enhancement in the performance of the model with transfer learning can be measured in three ways, with definitions based on the learning curve (model performance vs amount of training) [16]:

- initial performance (before the training),
- the slope of the learning curve (how fast the learning is completed),
- the final performance (the asymptote level).

One of the main advantages of transfer learning over isolated task learning is that it relaxes the requirement on the amount of training data for the target task. Instead, it benefits from the knowledge gained from a related task, for which more extensive datasets could be available [14, 17].

2.2.3 Classification of transfer learning methods

Approaches to transfer learning in the literature can be grouped into two main categories, depending on how the knowledge transfer occurs: on data or model level [14, 15]:

- Data-level:
 - **Instance-based approaches** [18, 19] - applying weights (*reweighting*) to samples in order to adjust the marginal probability distributions of the two domains;
 - **Feature-based approaches: asymmetric** [20, 21, 22] and **symmetric** [23, 24] feature transformation - aiming at minimising the disparity between feature spaces (in heterogeneous transfer) or marginal and conditional probability distributions (in homogeneous problems);
 - **Relational-based approaches** [25, 26], exploiting similarities in relationships present in the two domains.
- Model-level:
 - **Parameter-based approaches** [27, 28, 29] - based on the assumption that the parameters (and/or structure) of a model trained on the source domain can be re-used in target domain, with or without adjustments (fine-tuning).

2.2.4 Deep transfer learning

Deep learning algorithms are known to rely strongly on extensive amounts of training data, which may sometimes not be available. Thus, deep transfer learning has attracted major attention [14, 17].

There are several types of strategies to implement deep transfer learning, mostly following the classification presented in section 2.2.3. Multiple methods involving the manipulation of the training data have been proposed [17]. Domain adaptation itself can also be performed using neural networks - in particular, following adversarial approaches [17, 30, 31]. However, the principles of neural network training processes also make parameter-based approaches well applicable [17, 32].

The deep transfer learning process consists of [32]:

1. Training (*pretraining*) the base deep learning model on the source dataset and task,
2. *Repurposing* or *transferring* some of the features (weights) from the base to the target model,
3. Training the target model on the target dataset and task.

The target model is usually constructed by assembling the first n layers of the base model and several randomly initialised layers to be trained for the target task. The copied layers can either be *frozen* (weights are not allowed to change during the target training process) or set as *fine-tunable* (weights can be adjusted during the training). The fine-tuning strategy could improve the overall performance; however, it is likely to cause overfitting if the target dataset is relatively small [32].

In general, the initial layers of a neural network are observed to learn general features (e.g. Gabor filters equivalents for images), while the final layers are specific to the task. The number of transferred layers should be tuned, taking into account the dissimilarity between the source and target tasks [32].

Chapter 3

Literature Review

3.1 Deep learning for computer vision

3.1.1 Deep learning architectures for image processing

The basic CNN architecture has been used as a backbone on which to build more sophisticated image processing models. Many of them emerged as entries to computer vision competitions, such as ImageNet Large Scale Visual Recognition Challenge (ILSVRC), run annually from 2010 [33]. A core part of the competition is a high-quality, publicly available dataset of over 1.2 million images assigned to 1000 categories.

One of the early architectures which attracted major, long-lasting attention from the data science community was a network developed by Krizhevsky *et al.* in 2010 and 2012, later referred to as AlexNet [34, 35]. The network design contained five convolutional and three fully connected layers, accounting for 60 million parameters. Techniques such as dropout were used to reduce overfitting, which was reported to be a significant issue due to the extensive size of the network [34].

In 2014, Szegedy *et al.* proposed an architecture known by the name of GoogLeNet. The network had more layers (22), but fewer parameters (about 5 million) than AlexNet. Its remarkable feature was *inception architecture*, implying that the building blocks of the network were smaller network modules (*inception modules*); the total number of independent layers was higher (ca. 100) [36, 37].

Several other particularly well-performing models have been developed since then. Some of the most popular ones are summarised in Table 3.1 below.

3.1.2 Computer vision with deep transfer learning

In recent years, many projects involving image recognition have resorted to transfer learning to improve the performance achieved. In some cases, a model is first trained on an open dataset, such as ImageNet, and then re-purposed for the target task [23]. However, and particularly in more recent projects, ready, pretrained models are increasingly being adopted [42].

3.1.2.1 Adjustments to the target dataset

As a first step, the target dataset must be adjusted for use with the chosen model. This frequently involves resizing the images to fit the required input shape (defined as image width, height, and number of channels) [35, 42]. Additional domain-specific preprocessing techniques could also be applied to enhance the results [42].

Table 3.1: Summary of pretrained CV models (conv. - convolutional, fc. - fully connected layer)

Model	Developed by	Year	Description
AlexNet [34]	Krizhevsky <i>et al.</i>	2010 -2012	8 layers (5 conv., 3 fc.); 60 M parameters; ReLu activation
GoogLeNet [36]	Szegedy <i>et al.</i>	2014 -2015	22 layers; 5 M parameters; <i>inception architecture (network in network)</i>
VGG-16 [38]	Simonyan & Zisserman	2014	16 layers: 13 conv., 3 fc.; 138 M parameters
VGG-19 [38]	Simonyan & Zisserman	2014	19 layers: 16 conv., 3 fc.
ResNet-50 [39]	He <i>et al.</i>	2016	up to 152 layers, 26 M parameters; using batch normalisation; <i>shortcut connections</i> skipping layers to learn residual mapping
Xception [40]	Chollet	2016	22 M parameters; based on Inception model; <i>depthwise separable convolutions</i> - separate spatial and cross-channel correlation
MobileNet [41]	Sandler <i>et al.</i>	2017 -2018	1.7 - 6.9 M parameters; <i>linear bottleneck</i> layers to learn residuals

One of the issues to be considered is “data capture bias” [23]. For example, the standard source datasets, such as ImageNet, often depict single, distinct objects, easily distinguishable from the background. Real-life examples from target tasks, on the other hand, usually contain multiple objects of different relative sizes, and frequently only object parts are visible. This could be dealt with by performing *scanning window search* on the target images to extract patches potentially containing objects of interest [23, 43]. Depending on the task, the scanning could be realised at different scales, taking into account domain knowledge or at random [23, 35].

In some studies, issues concerning the graphics type (e.g. photos, drawing, cartoons, X-ray scans) have been raised [44, 45, 46, 47]. However, transfer learning with networks pretrained on ImageNet or similar datasets has been demonstrated to perform well in a number of scenarios, including medical imaging of different types [35, 46, 48]: X-ray luggage scans [43], industrial equipment and construction screening [49], and spectrograms of physical phenomena in microscale (e.g. *glitch* artefacts from LIGO detectors) [42].

3.1.2.2 Using pretrained models and adapting the output

Once the pretrained model for transfer learning is chosen, the next key decision concerns how to implement it in the training process. The general approach is to use a selected (major) initial portion of the model [32]. One possible approach is to use the feature extraction capability of the pretrained network and continue classification with an alternative model [50]. In many cases, particularly for the more complex architectures, the full model structure is reused [35]. One of the most frequent and natural adjustments is replacing the output layer with one adapted for the specific problem of interest, e.g. suited for the number of classes in the target task [42]. Multiple *adaptation layers* could also be added in advance of the final layer to improve adjustment to the target domain [23]. Other possible modifications include adding dropout layers before and/or after the pretrained model to reduce overfitting [42].

Another question to be considered is whether to allow retraining of the learned parameters - and if so, at what rate and with how many layers. In the simplest, *off-the-shelf* setting, the majority of the layers are frozen only to retrain the last, classifier layer [35]. In case there is a larger disparity between the source and target domains, the full models are fine-tuned using the target dataset [42]. Many studies adopt a hybrid approach, where most layers of the model are fine-tuned with a decreased learning rate (e.g. by the factor of 10) and the final ones are trained from scratch [35, 51, 52, 53]. It has been demonstrated that while more fine-tuning improves accuracy for larger datasets, smaller ones benefit from more layers being frozen [53].

3.2 Deep learning for natural language processing

3.2.1 NLP with deep transfer learning

In recent years, multiple variants of deep learning architectures for NLP have emerged. Many of them have been created with transfer learning in mind. Instead of constructing a model for a specific task, the focus has been to design a general-purpose architecture, which could be fine-tuned to various target problems.

One of the earliest such models is Universal Language Model Fine-tuning (ULMFiT), proposed by Howard & Ruder in 2018 [54]. ULMFiT is based on a simple LSTM architecture with dropout hyperparameters. The paper focused on the fine-tuning process, proposing several strategies:

- *discriminative fine-tuning* - with different learning rates for each layer;
- *slanted triangular learning rates* - first linearly increasing, and then decaying learning rates over the course of re-training;
- *gradual unfreezing* - unfreezing the layers gradually in consecutive fine-tuning iterations, starting from the final ones.

ULMFiT was pretrained on WikiText-103 dataset [54, 55] and showed greatly improved performance for multiple text classification tasks [54].

Another famous development in recent years is BERT - Bidirectional Encoder Representations from Transformers [56]. The main idea of BERT is to extract text representations considering context in both directions - left and right. This was a significant advancement in comparison to standard Transformers architectures, which are limited to single-side context (words either preceding or following the term of interest).

BERT is trained in two stages: pretraining and fine-tuning. The pretraining dataset does not have to be annotated; it is automatically transformed to form a supervised learning problem. There are two types of pretraining tasks for BERT:

- Next Sentence Prediction - classification whether a sentence would logically fit as subsequent to another given sentence,
- Masked Language Modelling - finding a word which would fit best into a gap in a sentence.

BERT was originally pretrained on English Wikipedia and BookCorpus datasets, which provide *document-level corpus* [56]. Multiple sizes of BERT model have been made available, including *base*, *large*, and *smaller* [57].

In many studies, BERT was subjected to further pretraining to adapt it to specific domains. Examples include: SciBERT for scientific corpora [58], BioBERT for biomedical texts [59], CT-BERT (COVID-Twitter-BERT) for COVID-related Twitter posts [60], and CamemBERT for the French language [61]. Alternative, *robustly optimized* pretraining methods were used in RoBERTa [62].

The fine-tuning of BERT and BERT-derived models involves wrapping the pretrained body of the model into a desired structure (e.g. a classifier) and executing the training using the target dataset. According to the original paper, a single additional output layer is sufficient to produce high-accuracy results across a variety of NLP tasks [57].

Transformer-XL (*extra-long* Transformer), proposed by Dai *et al.* in 2019, expands the idea of Transformer architecture [63]. It has been designed with a focus on long-term dependencies not limited by a fixed-length context, which is the case for *vanilla* Transformers. Transformer-XL architecture is based on recurrent connections, which enable the transfer of information over large *distances* and hence the modelling of long-term dependencies. What is more, the use of recurrence was shown to markedly boost the evaluation speed - by up to 1800 times as compared to basic Transformer architecture [63]. Additionally, Transformer-XL introduced a novel type of *relative positional encoding* method. It facilitates the long-term dependency information passage and solves the associated performance issues posed by absolute positional encodings [63].

3.2.2 Datasets for NLP with transfer learning

Table 3.2 presents a summary of some of the most popular datasets frequently used for pretraining language models.

Dataset	Size (words)	Description
Penn Treebank [64]	4.5 million	English texts extracted from the Wall Street Journal (1989-1992), with part-of-speech annotation
WikiText-2 [55]	2 million	Pre-processed Wikipedia articles marked as <i>Good</i> or <i>Featured</i>
WikiText-103 [55]	103 million	
English Wikipedia [57]	2.5 billion	Text blocks from Wikipedia (no tables, lists, etc.); used in BERT [56]
BookCorpus [65]	800 million	Corpora extracted from 11 thousand books of 16 genres, available online
Billion Word Benchmark [66]	1 billion	English corpora extracted from WMT11 website [66]; shuffled sentences, not document-like corpus [56]

Table 3.2: Examples of the most popular NLP datasets

3.3 Deep learning for hybrid tasks

In many real-life problems, information is not limited to a single particular form, such as numerical time series, text, images, etc., but is a combination of these. As described in [67], there are three main types of approaches utilised in multi-modal machine learning:

- *early fusion* - combining all data modalities into a feature vector, fed as a single input to an ML algorithm
- *late fusion* - processing the data modalities separately and making the prediction based on combined outputs;

- *intermediate fusion* - combining the modalities at an intermediate stage; e.g. in neural networks: developing initial separate pathways for the modalities, binding the results together, and adding more layers on top of the ‘junction’.

The latter solution addresses the limitations of both early fusion, which disregards the different natures of the modalities, and late fusion, which assumes the modalities to be uncorrelated [67].

Various strategies for implementing intermediate fusion have been proposed. Zadeh *et al.* developed a *Tensor Fusion Network* (TFU), which captured *intra-modality* and *inter-modality* information by computing an outer product between the different representations of the respective modalities [68]. For purely sequential data, a *Memory Fusion Network* (MFU) was proposed, utilising a *cross-view relevance score* to represent the interdependencies between the different modalities. [69]. In *Deep Multimodal Attentive Fusion* architecture, proposed by Huang *et al.* a multi-layer perceptron module was used on top of a text-image modalities junction to process the non-linear inter-modality dependencies [67]. The final prediction in this case was derived by applying late fusion on the results of three submodels: image, text, and joint text-image model, to account for data incompleteness.

3.3.1 Multi-modal deep learning for social media data

A particularly abundant source of multi-modal learning problems are various social media platforms, where post text is regularly accompanied by an image (or vice versa if the platform’s focus is predominantly visual rather than textual). One type of tasks involving joint image and text data is the classical problem of sentiment analysis. In [70], a bimodal text-image model was developed for the analysis of the SentiBank dataset, which contains images collected from Flickr labelled by adjective-noun pairs (ANP). The model was then transferred to predict sentiment for Twitter data. Huang *et al.* developed a bimodal sentiment analysis network for images with associated English texts (descriptions, tweets, keywords), extracted from the Getty Image, Twitter, and Flickr platforms [67]. The performances of several different models were compared, including unimodal architectures as well as early and late fusion schemes. In [71], Zhao *et al.* explored the significance of consistency between image and associated texts (e.g. accounting for sarcastic content) in SentiBank Flickr data, focusing on posts where text is embedded in images. Zadeh *et al.* assessed sentiments from movie reviews published on YouTube [68]. The analysis used up to three subnetworks dedicated to specific input types: spoken language, visual information (extracted using a facial expression analysis framework), and various acoustic features (related e.g. to voice pitch and volume).

Sentiment analysis is not the only popular topic in social-media-data-based research. Many projects focus on examining the general climate of opinion in society, which is well reflected in social media. Yazdani *et al.* analysed Twitter posts with non-photographic images - in particular, images of text - to predict various socio-economic trends [72]. The analysis included geographical location (limited to 20 cities in the US) and features inherent to social media, such as the re-sharing rate. In [73], a multi-modal deep learning system was developed for the identification of collective action events (e.g. protests) on Chinese social media platforms - in particular, a microblogging service *Sina Weibo*. Ajao *et al.* developed an LSTM-CNN model to detect fake news on Twitter [74]. The target posts included those posted by verified accounts after having been hacked by malicious organisations.

Mouzannar *et al.* proposed a system for screening social media for *actionable information* in crisis situations, such as natural disasters [75]. The developed model performed classification of Instagram and Twitter posts (including images and texts) to identify those which might help address the issues more efficiently e.g. by better allocation of required resources. Within the multi-modal architecture, the work used pretrained models to process the visual and textual inputs.

3.3.2 Image-to-text generation

A related type of setting where multi-modal learning has been shown to be useful is for a range of image-to-text generation tasks. One of the most common applications is in automated image captioning. In its basic version, a CNN and an RNN are used sequentially: the CNN as an encoder to extract a *visual feature vector* from an image and the RNN as a decoder to formulate image description based on the former [76]. Multiple modifications to the outline architecture have been proposed to further improve model performance [76, 77, 78, 79].

Image-to-text generation models have also been applied to the field of computer-aided diagnosis (CAD), where they are used to aid in recognising pathological conditions e.g. in X-ray and MRI scans [80, 81]. In an article by Wang *et al.* [78], CNN and LSTM models were used to develop a Text-Image Embedding network for the classification of a range of thoracic diseases. During training, the model used chest X-ray scans and associated textual reports to automatically extract disease names and identify their visual signs. The trained model was capable of performing multi-label disease classification based on single X-ray images.

Chapter 4

Methods

This chapter describes the development of *Instarget* - a deep learning system for classification of posts on Instagram social platform with respect to business relevance, i.e. for identification of *Instagram business search targets* or *business signals*. The system was developed from the bottom up, including the construction of a data acquisition setup, data preprocessing, model definition and training, and evaluation tools.

The implementation described in this chapter was prepared in the Python programming language. The back-end of the system was developed as a Python package. Some parts (the front-end for data collection and final pipeline construction) were prepared in Google Colaboratory (Colab). The deep learning model was defined and trained using a TensorFlow framework with Keras back-end; some components of the system also used Huggingface Transformers library with Tensorflow compatibility. Additional resources used in the project are listed in Instarget package requirements. User manual for the system can be found in Appendix E.

4.1 Data

4.1.1 Data acquisition

As there was no annotated dataset available for the project, a data acquisition system was prepared.

4.1.1.1 Requirements

There were several requirements for the data acquisition setup to be built.

First of all, the data was to be annotated by DeepMiner's CEO, a person with expertise in the business field, but limited technical experience. Therefore, the primary prerequisites for the system were: user-friendliness (simple interface), usage readiness (requiring no installation), and fault tolerance.

The design had to also take into account that the annotation work would be realised over a prolonged period rather than a single session.

The examples were to be annotated in a two-level manner:

- binary classification: a post is or is not a business signal (*positive* and *negative* labels respectively),

- multi-label classification: business signal types (for posts classified as business signals in the first stage).

4.1.1.2 Instagram web scraper

The data acquisition framework was designed to perform data collection and annotation simultaneously (in contrast to two-stage solutions, where previously-collected data is later submitted for annotation). The setup consisted of two main elements: a back-end data extraction script and a Google Colab notebook front-end.

The data extraction from Instagram website was performed using a custom-built web scraper script, written in Python using the packages `selenium` (with Chrome browser driver) and `requests`.

The functionalities implemented in the scraper contributed to the following workflow:

1. Logging in to Instagram using credentials for a DeepMiner account, created for the purpose of this project.
2. Selecting the required number of the most recent posts from the list of Instagram profiles specified by the user.
3. Collecting metadata for each post (as described in detail in section 4.1.2).
4. Annotation stage: asking the user for labels for each post. For consistency purposes and in order to have the possibility to implement input validation, the user was asked to introduce integer codes corresponding to the labels rather than label names.
5. Downloading images associated with the posts and storing them in a location indicated by the user.
6. Storing the scraped data (including file names of the downloaded images) to a CSV file. If the user-specified file name matched an existing file, the scraper would merge the two files after a structural consistency check, retaining recently-acquired examples to help identify duplicates.

The scraper was designed to have two working modes:

- *head-on* - suited for standalone operation (i.e. executing locally from a computer with Python 3 installation and an appropriate working environment configuration); in this mode, the web driver was set to explicitly open a browser window for the user to be able to track all actions and visited sites;
- *headless* - intended for operation in combination with the aforementioned Google Colab setup, as described in detail in section 4.1.1.3 below; working without an explicitly open browser window (as this setup is not feasible on Google Colab servers, which are normally not equipped with screens); instead, the scraper would present the user with screenshots and hyperlinks to the pages of interest.

4.1.1.3 Google Colab data acquisition setup

The scraper front-end was created in Google Colab as this tool provided an optimal compromise between the user-friendliness and time efficiency of the development process. It did not require any

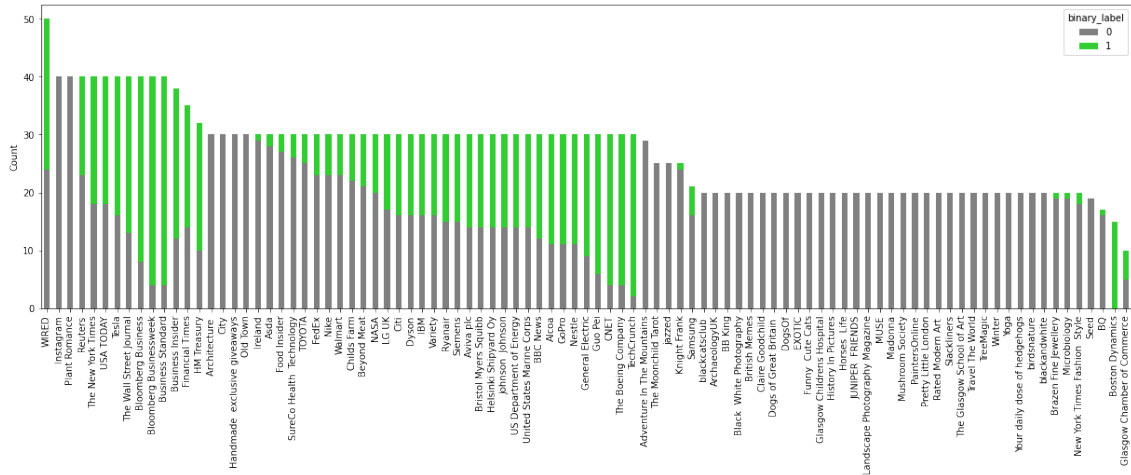


Figure 4.1: Positive and negative examples per profile

installation on local machines, which would be a barrier to the annotator. Moreover, the data could be stored directly in the Google Drive, which made it immediately accessible for analysis.

The scraper script, described in section 4.1.1.2, was made available for the Colab-based annotation tool by storing the data in a common location in Google Drive and *mounting* the Drive in Colab. The user is asked to specify the names of the Instagram profiles they wish to extract the data for, as well as the number of posts for each profile, prior to commencing annotation. Running the notebook executes several automatic setup operations, after which the annotation process begins. The user is presented with a sequence of screenshots of Instagram posts to annotate - one at a time - together with links to access them directly. Once an integer label code is entered, the program proceeds to the next post, until the end of the previously specified set.

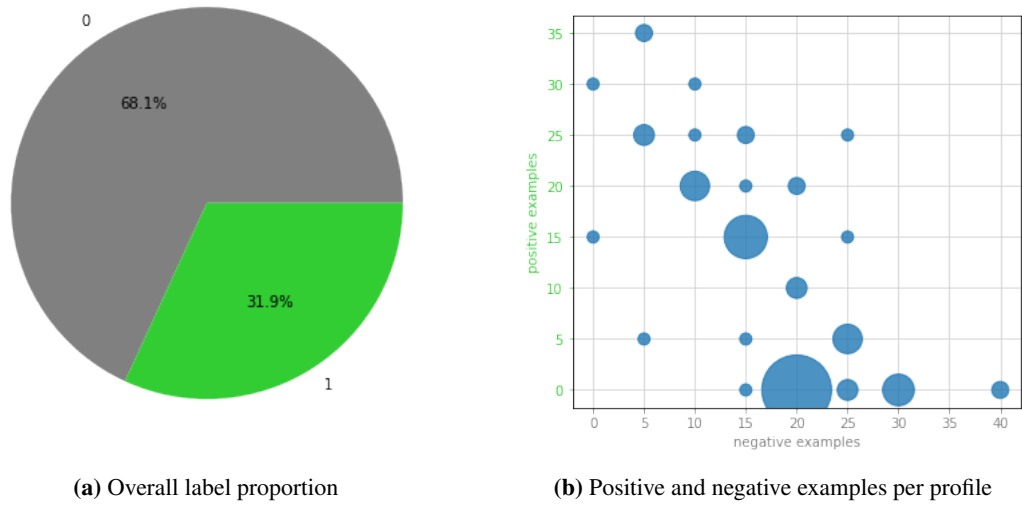
The full usage documentation of the Google Colab-based annotation tool as created for this annotation task is included in the Appendix B.1. The notebook itself with data annotation examples is also included in Appendix B.2.

4.1.1.4 The data collection process

The data was collected and annotated by DeepMiner’s CEO and CTO over the period of 9th to June 24th July 2020. Ten to fifty of the most recent posts were extracted from a number of Instagram profiles sharing potentially relevant business information. The profiles to be processed were selected by the CEO and CTO. Additionally, a number of clearly negative examples (posts from non-business accounts) were acquired by the researcher. A detailed description of the collected data is presented in section 4.1.2.

4.1.2 Description of data collected

The collected dataset contained 10 to 50 examples extracted from 93 profiles - in total, 2511 samples. The number of positively- and negatively-labelled examples per profile is presented in a bar plot in Fig. 4.1. A full-sized version of this plot is available in Fig. C.1 (Appendix C). Fig. C.2 depicts the numbers normalised to the total number of examples per posts (in percentages per profile). In all plots related to binary labels, the positive examples (labelled as 1) are represented in green, and the negative (label 0) - in grey.

**Figure 4.2:** Caption

The total numbers of positive and negative examples were 800 and 1711, i.e. 32% and 68% of the dataset, respectively, as depicted in Fig. 4.2a. As shown in Fig. 4.1, several profiles significantly contributed to the 1:2 proportion by introducing only negative examples. They were included to approximately reflect the imbalance observed in real-world data - the majority of Instagram posts is expected not to be of business relevance.

Fig. 4.2b presents the numbers of positive and negative examples per profile on a grid in the form of a bubble plot. The post numbers - coordinates in the plot - have been rounded to the nearest multiple of 5. Profiles with the same coordinates have been aggregated to form larger ‘bubbles’. For example, there were many posts with approximately 20 negative and no positive examples, as well as those with a 50-50 label split (found on the main diagonal of the chart).

For each sample, the collected data consisted of the following attributes:

- binary label - 0 or 1
- multi-class label - a sequence of strings denoting adhesion to different business signal classes (for negative examples - an empty sequence)
- user name - an Instagram user name, e.g. ‘teslamotors’ or ‘glasgow_chamber’
- profile name - an Instagram profile name, e.g. ‘Tesla’, ‘Glasgow Chamber of Commerce’
- verified profile indication (Boolean value) - whether a profile has been assigned a *verified* badge
- time stamp - date and time of posting
- post URL - a link to the original post
- source URL - a link to download the cover image associated with the post¹
- description - text associated with the post

¹In cases where multiple images were included in the post, only the first one was considered; for video posts, the display cover - which appears before the video plays - was used.

- alternative text - extracted from the ‘alt’ attribute of the HTML ‘img’ tag²
- number of likes (for videos, the number of views).

Example dataset entries (excluding source URL) are shown in Tables C.1 and C.2 (Appendix C). The text length displayed was limited to 300 characters (in some cases, the descriptions were significantly longer). Non-Unicode characters from descriptions and alternative texts were removed when the data were stored.

The cover images associated with the posts were downloaded from the source URLs and stored in JPEG format in the Google Drive. The relative locations of the image files were recorded in the dataset (saved as a CSV file). All the images were of square shape with 3 colour channels (RGB); the resolutions of the images varied.

4.2 Data processing pipeline

In this section, the consecutive steps for processing the collected data are described. This includes preprocessing and feature extraction, definition and training of the deep learning model, and evaluation of the results.

4.2.1 Preprocessing and feature extraction

The definition of the task put the focus on visual and textual data, which also comprised the main input for the model. As described in section 4.2.2, the deep learning classifier included two pretrained models: Xception for image processing and RoBERTa for textual analysis. The preprocessing methods applied, described in this section, were designed to adjust the data to the input requirements for the aforementioned pretrained models.

4.2.1.1 Image preprocessing

The loading and preprocessing of the images was performed using functionalities provided by the TensorFlow library. On loading, the images were decoded to integer tensor (valued [0-255]) and converted to float values ([0-1] range). Next, they were resized to 224 x 224 x 3 in order to comply with the required input shape for the Xception model. The image preprocessing procedure was prepared based on an official TensorFlow example [82] and is illustrated by the following code snippet.

```

1 import tensorflow as tf
2
3 IMG_SIZE = 224
4
5 def load_image(file_path):
6     return decode_img(tf.io.read_file(file_path))
7
8 def decode_img(img):
9     # 1) decode to uint8 tensor
10    img = tf.image.decode_jpeg(img, channels=3)
11
12    # 2) convert to floats in range [0, 1].
13    img = tf.image.convert_image_dtype(img, tf.float32)
```

²In some cases, the alternative texts were identical to post descriptions; however, they often included metadata, such as location, tagged profiles, and objects recognised automatically from the associated image.

```

14
15     # 3) resize to 224 x 224 (x 3)
16     img = tf.image.resize(img, [IMG_SIZE, self.IMG_SIZE])
17
18     return img

```

The preprocessed, resized images were stored in the Google Drive in place of the photos in the original format as downloaded. This limited the amount of storage space they took up and accelerated the process of loading the images later to the pipeline.

4.2.1.2 Post description preprocessing

There are several standard techniques used for preprocessing textual input for Natural Language Processing purposes. This includes removing *stop words*, lowercasing, tokenisation, and lemmatisation [83]. However, the RoBERTa model was pretrained on cased text, including stop words and punctuation, and the tokenisation process was based on the WordPiece model [56, 62].

To comply with the methods originally used with RoBERTa, only rough initial preprocessing was applied, after which a pretrained RoBERTa tokeniser from the Transformers library [84] was used to tokenise and encode the input. The full post description preprocessing workflow consisted of the following operations:

1. Removing links (Instagram-specific tags and hashtags were retained),
2. Inserting separation tokens (</s>) between sentences (the pretrained tokeniser added separation tokens only at the ends of the entire text sequence),
3. Tokenising, padding or truncating, and encoding the text sequences.

The first two steps were achieved using regular expressions, as illustrated below.

```

1 import re
2
3 # link pattern: starts with 'www.' or 'http'/'https', no whitespaces
4 LINK_PATTERN = r'((www\.[^\s]+)|(https?:\/\/[^\s]+))'
5
6 # sentence boundary pattern: punctuation sign followed by whitespace
7 # and an alphanumeric sequence
8 SEP_PATTERN = r'(?P<interp>[\.!\?])\s+(?P<nextword>[w+])'
9 SEP_TOKEN = '</s>'
10
11 def prepare_line(text_line: str):
12     # remove links
13     re.sub(LINK_PATTERN, '', text_line)
14
15     # insert separation tokens
16     text_line = re.sub(SEP_PATTERN, sep_inserter, text_line)
17
18     return text_line
19
20 def sep_inserter(match):
21     # insert a separation token between sentences based on a regex match
22     return f"{match['interp']} {SEP_TOKEN} {match['nextword']}"

```



Figure 4.3: An example post with image-embedded tags by Knight Frank

The third preprocessing step was performed using the `batch_encode_plus` method from the pretrained RoBERTa tokeniser, provided by the Transformers library. The truncating and padding was applied in order for all the sequences to have the same length, matching the maximal input length of RoBERTa (512 characters). When truncating was required, the initial part of the text was preserved; it was assumed that the most relevant information can usually be found there (which was, however, only a partially-correct assumption; it was observed that in a small minority of cases the business-relevant information was mentioned only at the end of a long post).

The tokenisation output, produced by the RoBERTa tokeniser, consisted of two parts:

- `input_ids` - integer-encoded tokens from the processed text,
- `attention_mask` - an indication of positions of padding tokens, which the model should not consider during training.

Both inputs were later fed to the RoBERTa model in the full classifier.

4.2.1.3 Auxiliary text data preprocessing

In addition to processed input for the RoBERTa model, several other features were extracted from the available textual data - both post descriptions and alternative texts for images.

One of the characteristic features of Instagram posts is the use of hashtags (starting with '#', e.g. '#businessIntelligence') and tags, i.e. tagging other profiles by the user names (with the '@' prefix, e.g. '@teslamotors'). In post descriptions, many of the tags and hashtags appear at the end and could be discarded in description preprocessing for RoBERTa. Additionally, Instagram provides the option for embedding tags in images. Such tags are initially hidden and only become visible upon clicking on the post image. The tags in the image do not necessarily fully align with those mentioned in post descriptions. Examples of this are provided in Fig. 4.3 and 4.4.

The embedded tags could be extracted from the alternative texts for the posted image. For the post presented in Fig. 4.3 and 4.4, the alternative texts were as follows:

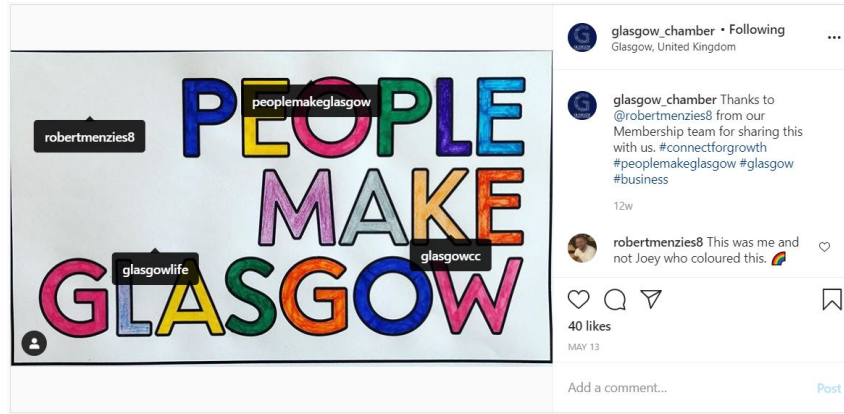


Figure 4.4: An example post with image-embedded tags by Glasgow Chamber of Commerce

‘Photo by Knight Frank in Central Park South, NYC with @knightfrankusa, @111west57st, and @nyproperties. Image may contain: sky and outdoor’

and

‘Photo by Glasgow Chamber of Commerce in Glasgow, United Kingdom with @peoplemakeglasgow, @glasgowlife, @robertmenzies8, and @glasgowcc.’

As shown in the examples, the alternative text may provide additional information on the location marked in the post and objects automatically detected in the images.

The post descriptions and alternative texts were used to extract these four types of information (hashtags, tags (tagged profiles), ‘may contain’ and location) using appropriate regular expressions:

```

1 # hashtags: '#' followed by Unicode word characters
2 HASHTAG_PATTERN = r'#(?P<hashtags>\w+)'
3
4 # tags: '@' followed by Unicode word characters with dots allowed
5 TAG_PATTERN = r'@(?P<tagged>[\w\.]*)'
6
7 CONTAIN_PATTERN = r'may contain:\s(?P<items>.+)'
8
9 # location: a combination of Unicode word characters, commas, whitespaces;
10 # found in posts where alternative texts are not copies of descriptions
11 # (normally starting with 'Photo [shared ]by');
12 # introduced by the word 'in'
13 # followed by tagged profiles ('with...') or an end-of-sentence dot
14 LOC_PATTERN = r'Photo (\w+\s+)*in\s(?P<loc>(\w+,?\s)*\w+)(?=( with|\.))'
```

The extracted sequences corresponded to the <hashtags>, <tagged>, <items>, and <loc> named groups. The prefixes '@', '#', 'may contain' and 'in' were not included.

The hashtags were additionally processed by splitting words represented in camel case and transformed them to lowercase. For example, hashtags extracted from the post shown in Fig. 4.5 were the following:

SmartCity, science, technology, AtlasOfDigitalization, CityOfTheFuture

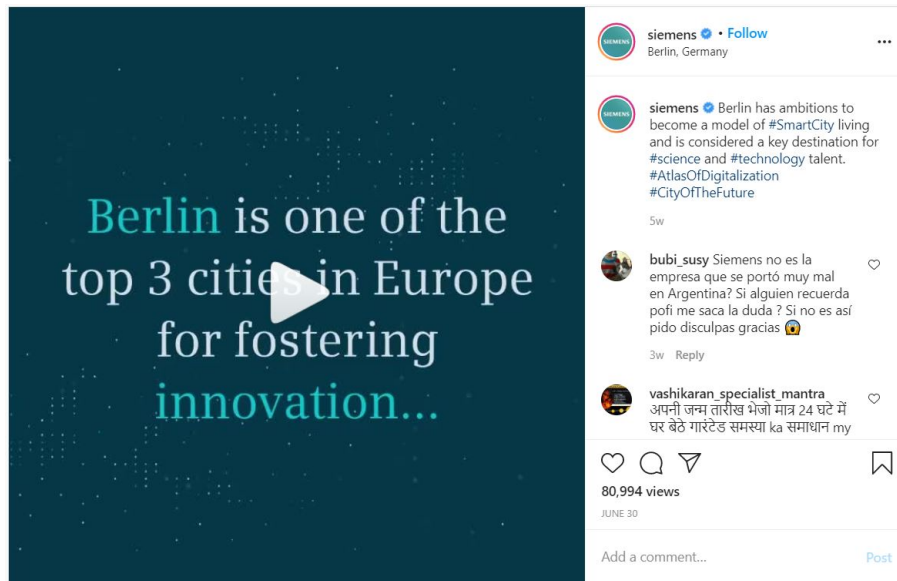


Figure 4.5: An example post with multiple hashtags by Siemens

After processing, the list was transformed into:

smart, digitalization, atlas, the, of, technology, science, city, future

which is significantly more generic than the original hashtags and therefore more likely to match other posts in the same topic area.

The extracted location and contained objects fields were tokenised by splitting on whitespaces. Subsequently, the four text types were encoded using the `hashing_trick` function from the `tensorflow.keras.preprocessing.text` module. The resulting sequences were padded with zeros to common lengths: 32 for tags and hashtags, 16 for contained objects, and 8 for locations

4.2.1.4 Feature definition summary

The full set of features - inputs for the classifier - contained four groups of parameters, as listed below. The names of each input as used in the model are included in Teletype font.

- image data, prepared as described in section 4.2.1.1 (images input)
- post descriptions, processed and encoded as in section 4.2.1.2:
 - `description_input_ids`
 - `description_attention_mask`
- auxiliary text data, following section 4.2.1.3:
 - `tags`
 - `hashtags`
 - `contains`
 - `location`
- additional numerical metadata (metadata input), consisting of two columns: verified profile indication and the number of image-embedded tags.

The collected dataset contained several additional types of information, such as profile name, number of likes assigned to the post, and date time. These data were not included in the analysis.

Profile name information was not used as the model was not to be dependent on profiles; instead, it should focus on the content. What is more, after deployment, the model would analyse posts retrieved from profiles it has not seen on the training stage. The information mismatch could impair the model's performance. The number of likes a post attracted is a feature partially dependent on the time since posting - the likes can be expected to accumulate over at least several hours, while DeepMiner would extract the posts shortly after them being shared. What is more, the absolute numbers of likes are strongly dependent on the profiles, information on which was excluded from the analysis. Finally, posting time was not regarded as a significant feature for assessing the relevance of the content.

4.2.2 Model definition

As mentioned before, there were two pretrained models incorporated into the structure of the classifier: a RoBERTa model for the processing of post descriptions and Xception for the analysis of the associated images. Additional sub-models were developed for auxiliary text features and numerical metadata, as described in section 4.2.1. In total, the complete model consisted of four submodels - 'branches', each exposing its own input (or inputs in case of the auxiliary text model). The outputs of the four submodels were then combined and topped with additional layers, including a classifier layer at the output.

Each of the model parts (the four input branches and the output) are described in further detail in this section. The full architecture of the model is summarised in a diagram in Fig. A.1 (Appendix A). Close-ups on two parts of the architecture are shown in Fig. A.2 (mostly the auxiliary text data branch) and Fig. A.3 (concatenation of the four input branches and combined output).

4.2.2.1 Xception input branch

The Xception model with weights trained on the ImageNet dataset was retrieved from `tensorflow.keras.applications` module. The classifier layer of the model was discarded:

```
1 from tensorflow.keras.applications import Xception
2
3 input_shape = (224, 224, 3) # rescaled square images with RGB channels
4
5 xception_model = Xception(input_shape=input_shape,
6                           include_top=False,
7                           weights='imagenet')
```

Following TensorFlow instructions [85], a global average pooling layer was added at the top of the model to transform spatial features into useful predictions.

The choice of the pretrained image processing architecture was performed in two stages. First, the models available from `tensorflow.keras.applications` (as presented at `keras.io/api/applications/`) were compared regarding their sizes and reported performance metrics. A few models with optimal combination in these two aspects were chosen. This included Xception, MobileNetV2, and InceptionV3. The performances of the different models on the sole image classification task (without additional features) were compared, and Xception was chosen as the one with best overall results to be included in the full model structure.

4.2.2.2 RoBERTa input branch

The RoBERTa model was used as made available from transformers library. The model class was sub-classed with overwriting the main layer call function (as described in a Tensorflow GitHub issue [86]) in order to enable the use of the functional TensorFlow model definition scheme:

```
1 from transformers import TFRobertaModel
2
3 class WrapRoberta(TFRobertaModel):
4     def __init__(self, config, *inputs, **kwargs):
5         super(WrapRoberta, self).__init__(config, *inputs, **kwargs)
6         self.roberta.call = tf.function(self.roberta.call)
```

The WrapRoberta model was initialised with roberta-base model (corresponding to the vocabulary file used to initialise a pretrained RoBERTa tokeniser, described in section 4.2.1.2). The model was treated as a single layer. It was connected to a double input (accepting the encoded input and attention mask). Out of the several outputs it produced, only one - pooled output - was used, in accordance with the Transformers library instructions and examples [87].

```
1 def make_roberta_layer(input_shape):
2     # prepare input layer (double input)
3     input_names = ['description_input_ids', 'description_attention_mask']
4     roberta_input = [Input(input_shape, name=name) for name in input_names]
5
6     # initialise RoBERTa
7     roberta_layer = WrapRoberta.from_pretrained('roberta-base')
8
9     # connect roberta layer to the input and pick the pooled output
10    # set 'training' to False to disable training of batch norm layers
11    # (see www.tensorflow.org/guide/keras/transfer_learning)
12    return roberta_layer(self._input, training=False)[1]
```

In the initial stage of the model development, a separate pipeline was built for analysis of textual data only. Two Transformers-based models were tested: RoBERTa and BERT. A range of different settings regarding the configuration of the added layers were explored with both models. The choice of the final structure was based on a comparison of the tendencies seen in the results, which were assessed using both quantitative and qualitative tools (e.g. analysing the particular misclassified examples).

4.2.2.3 MLP-Dropout block

The auxiliary text features branch as well as the output part of the model were constructed using an MLP-Dropout block structure. The block consisted of a number of fully connected layers with user-specified shapes, separated by dropout layers. In the next sections, the structure of the MLP-Dropout block is referred to by its dense layer configuration. For example, a 32x8 MLP-Dropout block consists of 3 layers: a dense 32-unit layer, a dropout layer, and an 8-unit dense layer.

All dense layers were L2-regularised with a regularisation factor of 0.001. The dropout rate on all added dropout layers in the model was set to 0.5. The only exception was a dropout layer on the output, before the classifier layer, where dropout rate was 0.2. The dropout rate scheme followed the standard setting introduced in the aforementioned original paper by Hinton *et al.*[6]. Different

settings of the dropout and regularisation rates were explored, but no significant improvements were seen for values other than the above mentioned standard.

4.2.2.4 Auxiliary text data input branches

The auxiliary text data model consisted of four branches - one branch for each text feature. Each of the branches consisted of the following stack of layers:

- an input layer adjusted to the encoded text length (see section 4.2.1.3);
- an embedding layer of size 128;
- a flattening layer;
- a 64x64x32 MLP-Dropout block; and
- a dropout layer (rate 0.5)

The outputs of the four branches were then concatenated and a 32x16 MLP-Dropout block was added at the top.

4.2.2.5 Metadata input branch

The final input branch consisted of an input layer followed by a single dense layer of size 4.

4.2.2.6 Combined model output

A dropout layer was added at the output of each of the four branches. The results were concatenated and a 32x32 MLP-Dropout block was added, followed by the last dropout layer (with dropout rate decreased to 0.2).

The model was finalised by adding a classifier layer: a fully connected layer consisting of a single neuron with sigmoid activation. The initial bias of the layer was adjusted to reflect the layer imbalance [88, 89].

4.2.3 Model training

4.2.3.1 Training epochs and eras

The model was trained following the principles underlying the use of pretrained models, as described in section 3.1.2.2. The complete training process could be divided into four ‘eras’, each consisting of multiple epochs.

First, the model was trained with both pretrained submodels frozen; only the weights of the added layers and the remaining two sub-models were updated. Next, the RoBERTa model was unfrozen and the model was trained at a lower learning rate. The third stage followed the gradual unfreezing procedure of Xception model. In each epoch, one more layer - starting from the top (output) of the Xception model - was allowed to update weights. In the final (10th) epoch of this era, the top 10 layers were unfrozen. This state was preserved in the final era of model training.

The process described above and key training parameters (learning rates and numbers of epochs) are summarised in Table 4.1 below.

The optimiser used in the process was Adam (from `tensorflow.keras.optimizers` module). Therefore, the learning rate specified in Table 4.1 should be interpreted as the initial value for the given era (with the exception of Era 3, where the 1E-5 rate was used in each training epoch).

Table 4.1: Training process of the Instarget binary classifier

Era	RoBERTa	Xception	Learning rate	Epochs
1	fully frozen	fully frozen	1E-3	15
2	unfrozen	fully frozen	1E-6	10
3	unfrozen	gradual unfreezing (one layer per epoch)	1E-5	10
4	unfrozen	top 10 layers unfrozen	1E-6	5

The aforementioned label imbalance was taken into account by utilising the `class_weight` parameter of the model `fit` method. Given the numbers n and p of negative and positive samples respectively, the weights w_n and w_p were calculated as follows:

$$w_p = \frac{n}{p+n}; \quad w_n = \frac{p}{p+n} \quad (4.1)$$

The formulas were derived based on two assumptions:

- The weights should reflect the inverse proportion between samples, i.e. $w_p/w_n = n/p$.
- The weights should sum up to 1.

The training process and data preparation, including the dataset split described in section 4.2.3.2, was performed with a fixed random seed to ensure the reproducibility of the results.

4.2.3.2 Data setup

The dataset for model training was constructed according to the description provided in section 4.2.1. It was then split into training, test, and validation sets in an 80:10:10 ratio; the numbers of samples in the resulting sets were 2009, 251, and 251 respectively.

The training dataset was used to train the model. The validation set was used to control the training process, including monitoring for possible overfitting, as described in section 4.2.3.3. The test set was held out during the training process and used to evaluate the model performance after completion of the training.

The datasets were cached, batched and prefetched. The last operation was introduced to let the dataset fetch batches in the background while the model was training and therefore optimise the process [90]. All the operations were performed using methods in `tensorflow.data.Dataset` objects:

```

1 import tensorflow as tf
2
3 def prepare_dataset(ds: tf.data.Dataset, batch_size: int):
4     ds = ds.cache().batch(batch_size)
5     return ds.prefetch(buffer_size=tf.data.experimental.AUTOTUNE)

```

The batch size for all datasets was set to 8. Using larger values was not feasible owing to hardware memory limitations.

4.2.3.3 Training process control

The training process was monitored using the validation set to prevent overfitting. Two types of training callbacks were implemented: early stopping and best model checkpoint.

The early stopping callback was set to monitor the loss of the model against the validation set. A training era was to be interrupted in case no improvement (i.e. decrease) in validation loss was seen in 5 consecutive epochs. In this case, the best weights of the model (i.e. those corresponding to minimal validation loss) were to be restored:

```
1 from tensorflow.keras.callbacks import EarlyStopping
2
3 early_stop = EarlyStopping(monitor='val_loss',
4                             patience=5,
5                             restore_best_weights=True)
```

The early-stopping callback was effective in the first two training eras (training added layers and fine-tuning RoBERTa), having 15 and 10 epochs respectively. In case of interruption, the next era would resume training on the best model from the previous era.

The third era consisted of 10 separated epochs; therefore, it was not possible to use early stopping in the same form as described above. Instead, a model checkpoint callback was defined to store the model's weights upon each performance improvement. The best weights were to be restored at the end of the third and fourth era.

During the development stage, it was observed that at the end of the training process the validation loss had a tendency to increase, while other parameters (accuracy, precision, recall) were still seeing improvements. Therefore, instead of loss, the checkpoint was set to monitor the validation accuracy:

```
1 from tensorflow.keras.callbacks import ModelCheckpoint
2
3 best_model_ckpt = ModelCheckpoint(MODEL_CHECKPOINT_PATH,
4                                   monitor='val_accuracy',
5                                   save_best_only=True,
6                                   save_weights_only=True,
7                                   verbose=0)
```

4.2.3.4 Hardware setup

The full data processing pipeline was executed in Google Colaboratory (Colab). The entire training process took approximately 90 minutes on a Tesla P100-PCIE-16GB GPU.

4.2.4 Evaluation methods

The model was evaluated according to a number of metrics, the values of which were monitored throughout the training process for both the training and the validation set. The metrics, as listed below, were defined using the following classes provided in `tensorflow.keras.metrics` module:

- binary accuracy,
- precision,

- recall,
- AUC (receiver operating characteristics area under curve),
- the number of false positives.

The history of the metrics values from the training as well as history of model loss were used to construct learning curves, which also aided in tuning the parameters of the model and the training process itself, which was performed in an iterative manner. The final model's predictions were used to construct confusion matrices and receiver operating characteristics (ROC) curves for the training and validation sets.

Additionally, misclassified examples from the validation set with predicted probabilities were presented to the DeepMiner CEO and CTO to assess the severity of the model's mistakes.

4.2.5 Storing the trained model

After completion of the training process, the model was stored to enable reusing it for generating predictions in the future. The Keras model structure and weights were stored using the `save` method of the model class. Additionally, the model wrapper used in the system, as well as a configured data preprocessor instance were saved as pickle files to ensure compatibility with the system. The full output had the following structure:

```
full_model
├── model_wrapper.pkl
├── preprocessor.pkl
├── wrapper.pkl
└── keras_model
    ├── saved_model.pb
    ├── assets
    └── variables
        ├── variables.data-00000-of-00001
        └── variables.index
```

Model and preprocessor restoring functions were also implemented in the model wrapper class to facilitate the loading process.

Chapter 5

Results

In this chapter, the performance of the model and the learning progress are assessed according to previously-defined evaluation methods.

5.1 Training process

The model learning progress was monitored throughout the training process by tracking six parameters: accuracy, precision, recall, AUC score, number of false positives (FP), and loss. The resultant learning curves are shown in Fig. 5.1. An enlarged version of the graph can be found in Fig. D.1 in Appendix D. Plots for the training set are shown in light blue (labelled as '<metric name>'), while the validation set is plotted in purple (labelled 'val_<metric name>').

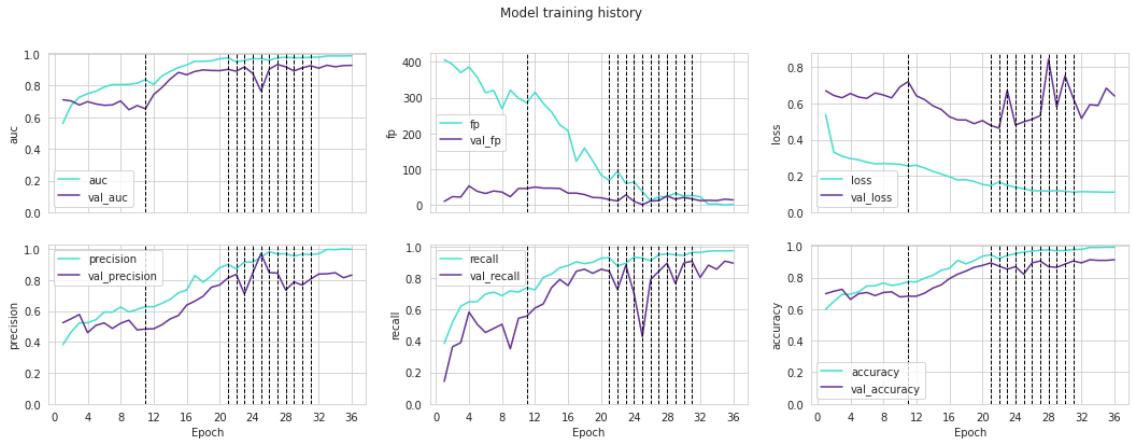


Figure 5.1: Model learning curves for the training and validation sets

Subsequent eras in the graphs in Fig. 5.1 are separated by vertical dashed lines. The densely crossed region corresponds to era 3 (gradual unfreezing of the Xception model), where the model was recompiled after each epoch.

As shown in the plots, the performance of the model gradually improved over the first era, tending towards a plateau. Unfreezing the RoBERTa model in the second era resulted in a surge in the performance on both sets, saturating approximately after 7 epochs. The gradual unfreezing of Xception model promoted further improvements. Significant fluctuations could be observed in the validation set metrics - with a positive overall tendency, however. In the final era, model recall on the validation set was enhanced, with minor changes in precision and accuracy.

As mentioned before, validation loss in eras 3 and 4 did not necessarily follow the trends observed in other metrics; for example, in the final era the loss clearly increased while accuracy, precision and recall still saw some improvements. This justifies the choice of validation accuracy as the criterion for choosing the best performing model.

The top validation accuracy over the entire training process was achieved in epoch 2 of era 4 (by a single false positive example better than the final epoch of this era). According to the earlier described methodology, weights of the model at that stage were restored at the end of training to produce the final model.

5.2 Trained model performance

The performance of the final trained model was assessed on the test set, which was not involved in any form in the training process. A full classification report for the test data is shown in Table 5.1. Table 5.2 compares the performance of the model on the three datasets. Confusion matrices are shown in Table 5.3.

Table 5.1: Classification report for the test set

	precision	recall	f1-score	support
0	0.96	0.93	0.95	180
1	0.84	0.90	0.87	71
accuracy			0.92	251
macro avg	0.90	0.92	0.91	251
weighted avg	0.93	0.92	0.93	251

Table 5.2: Comparison of model performance metrics on the training, validation, and test sets

	accuracy	precision	recall	auc
training set	0.99	1.00	0.97	0.99
validation set	0.91	0.84	0.88	0.93
test set	0.92	0.84	0.90	0.93

Table 5.3: Confusion matrices for training, validation, and test sets

		Pred.	
		neg	pos
True	neg	1357	0
	pos	17	635

		Pred.	
		neg	pos
True	neg	161	13
	pos	9	68

		Pred.	
		neg	pos
True	neg	168	12
	pos	7	64

The model learnt the training set almost perfectly, with all of the four metrics (accuracy, precision, recall, and AUC score) close to 1. As shown in the training set confusion matrix, there were no false positives; all negative examples have been correctly identified.

For validation and test set the metrics values are lower than for the training set, which may be indicative of some overfitting. However, they are still satisfactory, reaching over 80% for precision and recall and 90% for accuracy and AUC. According to all of the four metrics, the model's performance was very similar for the validation and test sets. The similarities can also be seen in ROC curves plotted in Fig. 5.2.

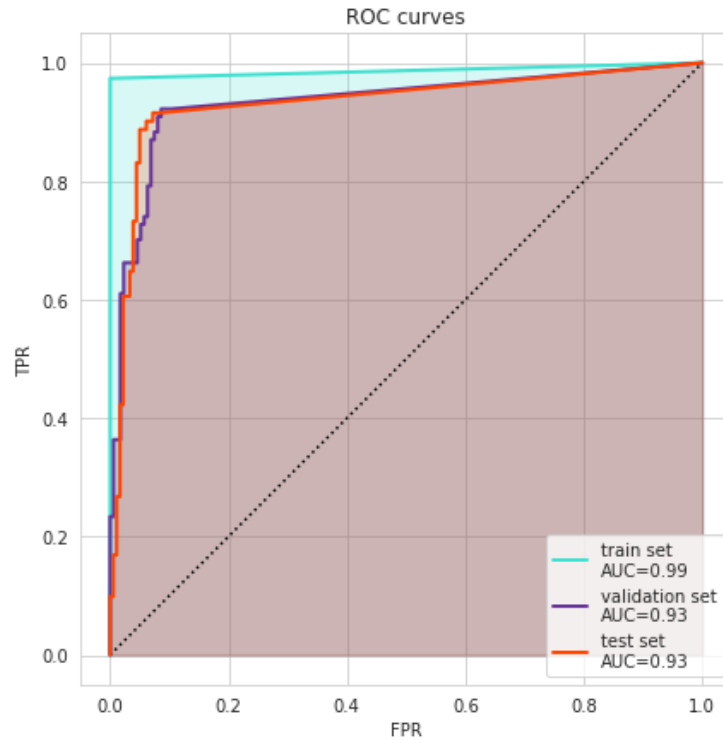


Figure 5.2: Receiver Operating Characteristics (ROC) curves of the model for the training, validation, and test sets; TPR - true positive rate, FPR - false positive rate

5.3 Qualitative assessment of the model performance

Examples of misclassified posts are shown in Fig. 5.3. The second line of each image title shows the true label, predicted label, and predicted probability (which could be interpreted as a post relevance score). An expanded version of the figure can be found in Appendix D (Fig. D.2). Associated textual data are shown in Table D.1.



Figure 5.3: Misclassified examples from the test set

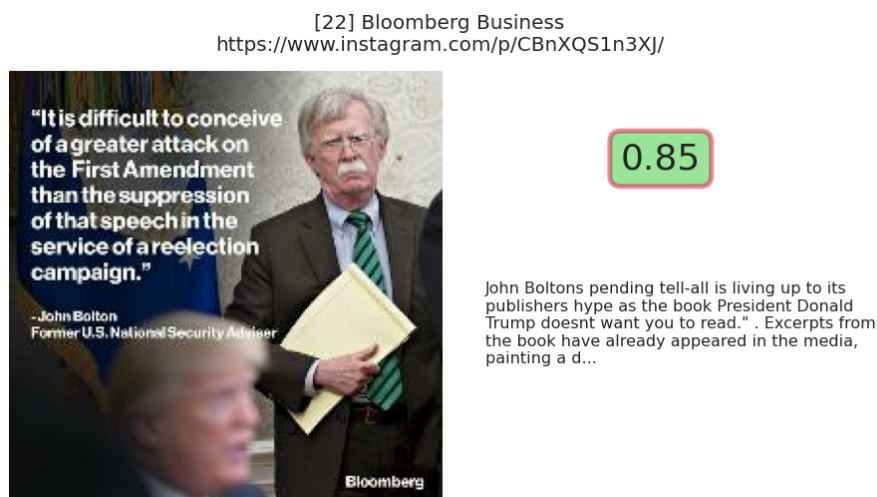


Figure 5.4: Misclassified post example 1; predicted score shown in the green rectangle

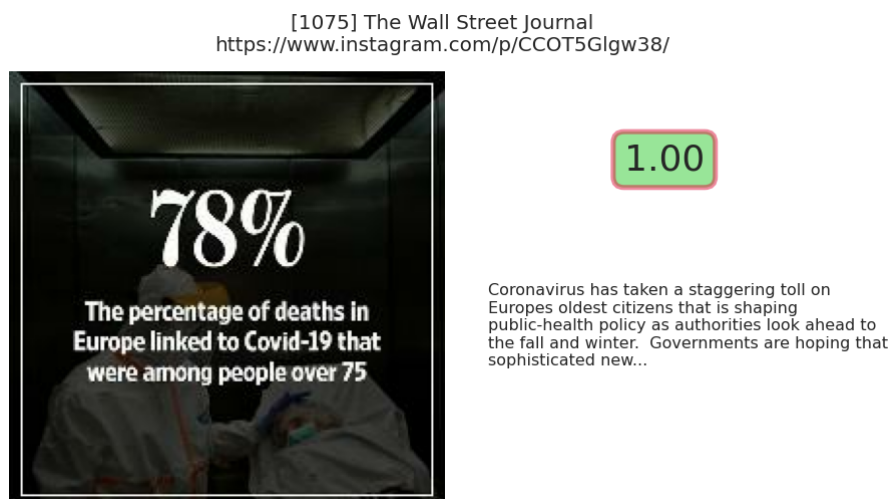


Figure 5.5: Misclassified post example 2

In many cases, the model predictions were close to the ground truth. For several of the misclassified examples, the relevance scores are close to 0.5. An additional observation is that the presence of text in post images often misleads the models. In the training dataset, text was embedded in many of the positive posts' images, which leads to high relevance scores for similar examples from the test set. This was likely the case for the post shown in Fig. 5.4 and 5.5

The test set data samples misclassified by the model have been presented to DeepMiner CEO and CTO, who assessed them on a four-level scale with levels: 'not an error', 'low', 'moderate', and 'severe'. The results are shown in Fig. 5.6.

As shown in the bar chart, none of the 21 errors the model made on the test set were termed as 'severe'. The two 'moderate' errors, both being false positives, are shown in Fig. 5.7 and 5.8. In both cases, the relevance score assigned to the posts by the model was very high (close to 1). In one of the cases, the post description contained the word 'teamwork', which could suggest some connections with the business area. This might have contributed to the model's decision, together with the 'text' recognised in the image - the number '33' on the runners' jerseys.

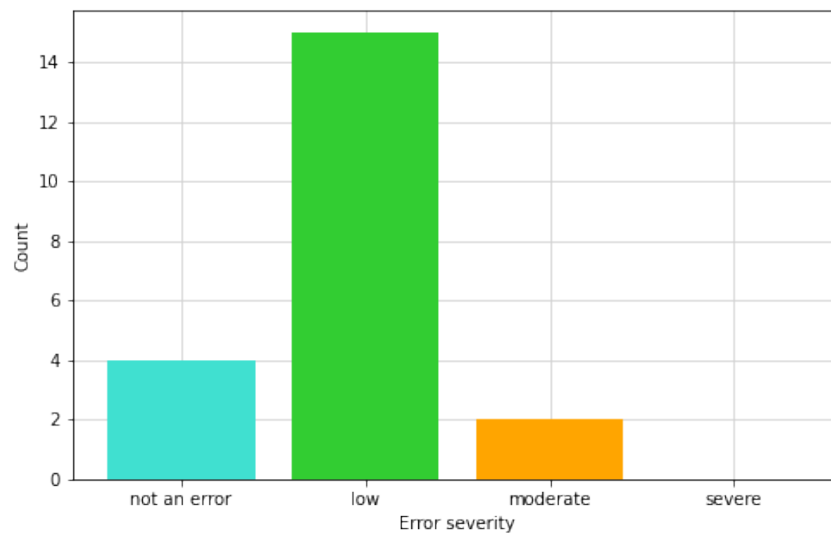


Figure 5.6: Severity of model mistakes on the test set

[1857] Johnson & Johnson
<https://www.instagram.com/p/BoM5yC2Fe0H/>



1.00

Theres teamwork, and then theres lacing up your running shoes and sprinting through the rocky terrain of Sweden before diving into the Baltic Sea off of the Stockholm Archipelago. And teamwork meant e...

Figure 5.7: 'Moderate' model error (1/2)

[1870] Variety
<https://www.instagram.com/p/CCwr3xDhs5L/>



1.00

This years #ActorsOnActors is unlike any other. Tune in to watch or stream the episodes tonight on @pbsocial at 8 p.m.

Figure 5.8: 'Moderate' model error (2/2)

Most of the model errors were of low severity. Many false negatives were in fact weak business signals. Some of them could be interesting to businesses, but further into the future, without concrete immediate implications. Others were relevant, but only to businesses in specific industries (e.g. an IBM post on the quantum technology they develop).

Many of the false positives, on the other hand, could be viewed as relevant to some degree, so presenting them to the user would not impair the noise-filtering functionality performance of the search engine as viewed by the end-users. In some cases, correct classification of the posts would require some general knowledge about the real world. As remarked by the DeepMiner CTO:

‘There are many business signals that can be hard to distinguish from empty marketing slogans and will only be shown to be proper signals if they are followed by actions.’

Upon reflection, the CEO and CTO agreed with some seemingly incorrect predictions of the model. Four of the twelve false positives were turned into true positives. This was the case e.g. for the previously discussed post shown in Fig. 5.5, as well as Fig. 5.9.

[34] The Wall Street Journal
https://www.instagram.com/p/CBIFM0ig_yS/



1.00

Three months after Perus president deployed soldiers to the streets in implementing one of the Western Hemispheres strictest coronavirus lockdowns, the country is grappling with Latin Americas worst o...

Figure 5.9: A false positive later assessed to be a true positive

Chapter 6

Conclusions

The work described in this dissertation combined multiple aspects of deep learning, using state-of-the-art techniques to create a fully functional product to answer the actual needs of industry. The developed system for the recognition of business signals in Instagram will serve as a business intelligence tool, extending the capabilities of the DeepMiner search engine.

Deep transfer learning was employed to achieve optimal results on a limited dataset. Many similar projects focus on re-purposing a pretrained NLP model for other NLP-type tasks, or fine-tuning an image classifier to adjust it to a custom set of classes. In contrast, this work combined two independent pretrained models - Xception for CV and RoBERTa for NLP - rated among the most successful developments of the recent years. A parallel combination of these with additional structures was designed to suit the multi-modal character of social media data.

One of the main goals of this work was to determine whether the automatic recognition of business signals - a concept which is nebulous to some extent - would be feasible. In this regard, the project was entirely successful. The developed model, trained on as little as 2000 examples, reached an accuracy level of 92%. The achieved precision and recall scores were both high (above 80%). The model adequately fulfilled its primary purpose of sifting out noise (accounting for the majority of Instagram posts) while remaining sensitive to actual business signals. What is more, the majority of the mistakes made by the model were of lesser importance.

While the experimental results were satisfactory, some aspects of the model development process could be changed. The structure of the model and the training process parameters were adjusted in an iterative manner, following a process of trial and error. The reproducibility of the results was mostly ensured by the fixed random seed. However, different seed settings - and therefore e.g. different training-test-validation dataset splits - were not systematically explored. A more favourable technique would involve adopting a cross-validation strategy using a number of parameters. Some additional adjustment of these could reduce model overfitting, which was still observable in the learning curves, and further improve the overall results.

An issue frequently raised with deep learning models is their lack of explainability. This is naturally the case in this work as well. The purpose of the product - facilitating the search for business opportunities - is not a question of life or death: in foreseeable uses, no danger to human health should be expected. On the other hand, the potential business users are more likely to mistrust a

model whose predictions are not easily explainable. Therefore, before model is incorporated into the DeepMiner search engine, it should be further evaluated on a larger dataset of new examples. The deployment in the live product should only be performed after making sure the unexplained error rates are low. Furthermore, techniques such as Local Interpretable Model-Agnostic Explanations (LIME) could be employed to investigate the nature of the model's mistakes.

6.1 Limitations

One of the limitations associated with the project was the assumption of binary character of business signals. Posts were initially expected to be either strictly relevant or irrelevant. However, further investigation has shown that this attribute should rather be considered in terms of a scale - a degree of relevance. Moreover, the relevance assessment is dependent on the end user - different types of information might be considered relevant for different people. In some cases, the DeepMiner CEO and CTO have different views regarding the correct labels. As discussed before, this ambiguity was a cause of some of the model's prediction errors. What is more, there were potentially more such examples in the training set, which might have introduced some confusion during the learning stage.

It should also be noted that the data acquisition setup was built from the ground up within the time frames of this project. What is more, the data collection process was extended over several weeks, during which it was sometimes required to provide technical help and solve unexpected issues. This limited the time available for the development of the model and full assessment of the performance of its consecutive iterations.

6.2 Future work

The model developed in this project will be deployed and incorporated into the existing DeepMiner search engine. The release is foreseen for several weeks after submission of this project - for September 2020.

There are several directions for possible future work after the initial release. One of the possible additions to the model is the classification of business signals into different categories. The dataset is already annotated to include the categories of interest. However, more examples might be needed as the positive class constituted only about 30% of the entire dataset.

The model in its current structure could be transferred to similar tasks on data retrieved from Instagram. Examples include sentiment analysis, as well as humour and sarcasm detection.

Some additional work could also be performed with respect to the preprocessing of data for the current model. In particular, the input to the RoBERTa submodel consisted of limited-length sequences (512 tokens) extracted from the beginnings of post descriptions. As mentioned before, the relevant information was sometimes included later in the text. One of the possible improvement strategies involves the development of a separate model to automatically select the most relevant fragment of the text; alternatively, an extract (summary) consisting of the most relevant sentences could be produced. Additionally, the usage of a different NLP architecture which accepts longer text fragments could be considered; however, this would likely come at a higher computational cost during model training and usage.

References

- [1] H. B. Demuth, M. H. Beale, O. De Jess, and M. T. Hagan, *Neural network design*. Martin Hagan, 2014.
- [2] S. S. Haykin *et al.*, *Neural networks and learning machines*. New York: Prentice Hall, 2009.
- [3] T. George Karimpanal and R. Bouffanais, “Self-organizing maps for storage and transfer of knowledge in reinforcement learning,” *Adaptive Behavior*, vol. 27, no. 2, pp. 111–126, 2019.
- [4] I. Goodfellow, Y. Bengio, and A. Courville, *Deep learning*. MIT press, 2016.
- [5] B. Ramsundar and R. B. Zadeh, *TensorFlow for deep learning: from linear regression to reinforcement learning*. O’Reilly Media, Inc., 2018.
- [6] G. E. Hinton, N. Srivastava, A. Krizhevsky, I. Sutskever, and R. R. Salakhutdinov, “Improving neural networks by preventing co-adaptation of feature detectors,” *arXiv preprint arXiv:1207.0580*, 2012.
- [7] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A simple way to prevent neural networks from overfitting,” *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014.
- [8] S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift,” *arXiv preprint arXiv:1502.03167*, 2015.
- [9] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer normalization,” *arXiv preprint arXiv:1607.06450*, 2016.
- [10] D. Ulyanov, A. Vedaldi, and V. Lempitsky, “Instance normalization: The missing ingredient for fast stylization,” *arXiv preprint arXiv:1607.08022*, 2016.
- [11] Y. Wu and K. He, “Group normalization,” in *Proceedings of the European conference on computer vision (ECCV)*, pp. 3–19, 2018.
- [12] T. M. Mitchell *et al.*, “Machine learning. 1997,” *Burr Ridge, IL: McGraw Hill*, vol. 45, no. 37, pp. 870–877, 1997.
- [13] S. Thrun and L. Pratt, *Learning to learn*. Springer Science & Business Media, 2012.
- [14] K. Weiss, T. M. Khoshgoftaar, and D. Wang, “A survey of transfer learning,” *Journal of Big data*, vol. 3, no. 1, p. 9, 2016.
- [15] S. J. Pan and Q. Yang, “A survey on transfer learning,” *IEEE Transactions on knowledge and data engineering*, vol. 22, no. 10, pp. 1345–1359, 2009.
- [16] L. Torrey and J. Shavlik, “Transfer learning,” in *Handbook of research on machine learning applications and trends: algorithms, methods, and techniques*, pp. 242–264, IGI global, 2010.
- [17] C. Tan, F. Sun, T. Kong, W. Zhang, C. Yang, and C. Liu, “A survey on deep transfer learning,” in *International conference on artificial neural networks*, pp. 270–279, Springer, 2018.

- [18] N. Jakob and I. Gurevych, “Extracting opinion targets in a single and cross-domain setting with conditional random fields,” in *Proceedings of the 2010 conference on empirical methods in natural language processing*, pp. 1035–1045, 2010.
- [19] J. Jiang and C. Zhai, “Instance weighting for domain adaptation in nlp,” in *Proceedings of the 45th annual meeting of the association of computational linguistics*, pp. 264–271, 2007.
- [20] S. J. Pan, J. T. Kwok, Q. Yang, *et al.*, “Transfer learning via dimensionality reduction,” in *AAAI*, vol. 8, pp. 677–682, 2008.
- [21] M. Long, J. Wang, G. Ding, S. J. Pan, and S. Y. Philip, “Adaptation regularization: A general framework for transfer learning,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 26, no. 5, pp. 1076–1089, 2013.
- [22] J. Hoffman, E. Rodner, J. Donahue, B. Kulis, and K. Saenko, “Asymmetric and category invariant feature transformations for domain adaptation,” *International journal of computer vision*, vol. 109, no. 1-2, pp. 28–41, 2014.
- [23] M. Oquab, L. Bottou, I. Laptev, and J. Sivic, “Learning and transferring mid-level image representations using convolutional neural networks,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 1717–1724, 2014.
- [24] S. J. Pan, I. W. Tsang, J. T. Kwok, and Q. Yang, “Domain adaptation via transfer component analysis,” *IEEE Transactions on Neural Networks*, vol. 22, no. 2, pp. 199–210, 2010.
- [25] Z. Yang, J. Zhao, B. Dhingra, K. He, W. W. Cohen, R. Salakhutdinov, and Y. LeCun, “Glomo: Unsupervisedly learned relational graphs as transferable representations,” *arXiv preprint arXiv:1806.05662*, 2018.
- [26] F. Li, S. J. Pan, O. Jin, Q. Yang, and X. Zhu, “Cross-domain co-extraction of sentiment and topic lexicons,” in *Proceedings of the 50th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 410–419, 2012.
- [27] Y. Yao and G. Doretto, “Boosting for transfer learning with multiple sources,” in *2010 IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, pp. 1855–1862, IEEE, 2010.
- [28] A. Rozantsev, M. Salzmann, and P. Fua, “Beyond sharing weights for deep domain adaptation,” *IEEE transactions on pattern analysis and machine intelligence*, vol. 41, no. 4, pp. 801–814, 2018.
- [29] L. Duan, D. Xu, and S.-F. Chang, “Exploiting web images for event recognition in consumer videos: A multiple source domain adaptation approach,” in *2012 IEEE Conference on Computer Vision and Pattern Recognition*, pp. 1338–1345, IEEE, 2012.
- [30] E. Tzeng, J. Hoffman, K. Saenko, and T. Darrell, “Adversarial discriminative domain adaptation,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 7167–7176, 2017.
- [31] Z. Cao, M. Long, J. Wang, and M. I. Jordan, “Partial transfer learning with selective adversarial networks,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 2724–2732, 2018.
- [32] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson, “How transferable are features in deep neural networks?,” in *Advances in neural information processing systems*, pp. 3320–3328, 2014.
- [33] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy,

- A. Khosla, M. Bernstein, A. C. Berg, and L. Fei-Fei, "ImageNet Large Scale Visual Recognition Challenge," *International Journal of Computer Vision (IJCV)*, vol. 115, no. 3, pp. 211–252, 2015.
- [34] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in *Advances in neural information processing systems*, pp. 1097–1105, 2012.
- [35] H.-C. Shin, H. R. Roth, M. Gao, L. Lu, Z. Xu, I. Nogues, J. Yao, D. Mollura, and R. M. Summers, "Deep convolutional neural networks for computer-aided detection: Cnn architectures, dataset characteristics and transfer learning," *IEEE transactions on medical imaging*, vol. 35, no. 5, pp. 1285–1298, 2016.
- [36] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich, "Going deeper with convolutions," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 1–9, 2015.
- [37] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the inception architecture for computer vision," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 2818–2826, 2016.
- [38] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," *arXiv preprint arXiv:1409.1556*, 2014.
- [39] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition. corr abs/1512.03385 (2015)," 2015.
- [40] F. Chollet, "Xception: Deep learning with depthwise separable convolutions, 2016," *arXiv preprint arXiv:1610.02357*, 2016.
- [41] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "Mobilenetv2: Inverted residuals and linear bottlenecks," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 4510–4520, 2018.
- [42] D. George, H. Shen, and E. Huerta, "Deep transfer learning: A new deep learning glitch classification method for advanced ligo," *arXiv preprint arXiv:1706.07446*, 2017.
- [43] S. Akçay, M. E. Kundegorski, M. Devereux, and T. P. Breckon, "Transfer learning using convolutional neural networks for object classification within x-ray baggage security imagery," in *2016 IEEE International Conference on Image Processing (ICIP)*, pp. 1057–1061, IEEE, 2016.
- [44] K. He, R. Girshick, and P. Dollár, "Rethinking imagenet pre-training," in *Proceedings of the IEEE international conference on computer vision*, pp. 4918–4927, 2019.
- [45] R. Geirhos, P. Rubisch, C. Michaelis, M. Bethge, F. A. Wichmann, and W. Brendel, "Imagenet-trained cnns are biased towards texture; increasing shape bias improves accuracy and robustness," *arXiv preprint arXiv:1811.12231*, 2018.
- [46] M. Raghu, C. Zhang, J. Kleinberg, and S. Bengio, "Transfusion: Understanding transfer learning for medical imaging," in *Advances in neural information processing systems*, pp. 3347–3357, 2019.
- [47] M. Lagunas and E. Garces, "Transfer learning for illustration classification," *arXiv preprint arXiv:1806.02682*, 2018.
- [48] A. Menegola, M. Fornaciali, R. Pires, F. V. Bittencourt, S. Avila, and E. Valle, "Knowledge

- transfer for melanoma screening with deep learning,” in *2017 IEEE 14th International Symposium on Biomedical Imaging (ISBI 2017)*, pp. 297–300, IEEE, 2017.
- [49] L. Attard, C. J. Debono, G. Valentino, M. Di Castro, A. Masi, and L. Scibile, “Automatic crack detection using mask r-cnn,” in *2019 11th International Symposium on Image and Signal Processing and Analysis (ISPA)*, pp. 152–157, IEEE, 2019.
- [50] M. Shaha and M. Pawar, “Transfer learning for image classification,” in *2018 Second International Conference on Electronics, Communication and Aerospace Technology (ICECA)*, pp. 656–660, IEEE, 2018.
- [51] A. Sharif Razavian, H. Azizpour, J. Sullivan, and S. Carlsson, “Cnn features off-the-shelf: An astounding baseline for recognition,” in *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, June 2014.
- [52] R. Girshick, J. Donahue, T. Darrell, and J. Malik, “Region-based convolutional networks for accurate object detection and segmentation,” *IEEE transactions on pattern analysis and machine intelligence*, vol. 38, no. 1, pp. 142–158, 2015.
- [53] M. C. Kruithof, H. Bouma, N. M. Fischer, and K. Schutte, “Object recognition using deep convolutional neural networks with complete transfer and partial frozen layers,” in *Optics and Photonics for Counterterrorism, Crime Fighting, and Defence XII*, vol. 9995, p. 99950K, International Society for Optics and Photonics, 2016.
- [54] J. Howard and S. Ruder, “Universal language model fine-tuning for text classification,” *arXiv preprint arXiv:1801.06146*, 2018.
- [55] S. Merity, C. Xiong, J. Bradbury, and R. Socher, “Pointer sentinel mixture models,” *arXiv preprint arXiv:1609.07843*, 2016.
- [56] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “Bert: Pre-training of deep bidirectional transformers for language understanding,” *arXiv preprint arXiv:1810.04805*, 2018.
- [57] I. Turc, M.-W. Chang, K. Lee, and K. Toutanova, “Well-read students learn better: On the importance of pre-training compact models,” *arXiv preprint arXiv:1908.08962*, 2019.
- [58] I. Beltagy, K. Lo, and A. Cohan, “Scibert: A pretrained language model for scientific text,” *arXiv preprint arXiv:1903.10676*, 2019.
- [59] J. Lee, W. Yoon, S. Kim, D. Kim, S. Kim, C. H. So, and J. Kang, “Biobert: a pre-trained biomedical language representation model for biomedical text mining,” *Bioinformatics*, vol. 36, no. 4, pp. 1234–1240, 2020.
- [60] M. Müller, M. Salathé, and P. E. Kummervold, “Covid-twitter-bert: A natural language processing model to analyse covid-19 content on twitter,” *arXiv preprint arXiv:2005.07503*, 2020.
- [61] L. Martin, B. Muller, P. J. O. Suárez, Y. Dupont, L. Romary, É. V. de la Clergerie, D. Seddah, and B. Sagot, “Camembert: a tasty french language model,” *arXiv preprint arXiv:1911.03894*, 2019.
- [62] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, “Roberta: A robustly optimized bert pretraining approach,” *arXiv preprint arXiv:1907.11692*, 2019.
- [63] Z. Dai, Z. Yang, Y. Yang, J. Carbonell, Q. V. Le, and R. Salakhutdinov, “Transformer-xl: Attentive language models beyond a fixed-length context,” *arXiv preprint arXiv:1901.02860*,

- 2019.
- [64] M. Marcus, B. Santorini, and M. A. Marcinkiewicz, “Building a large annotated corpus of English: The Penn Treebank,” 1993.
- [65] Y. Zhu, R. Kiros, R. Zemel, R. Salakhutdinov, R. Urtasun, A. Torralba, and S. Fidler, “Aligning books and movies: Towards story-like visual explanations by watching movies and reading books,” in *Proceedings of the IEEE international conference on computer vision*, pp. 19–27, 2015.
- [66] C. Chelba, T. Mikolov, M. Schuster, Q. Ge, T. Brants, P. Koehn, and T. Robinson, “One billion word benchmark for measuring progress in statistical language modeling,” *arXiv preprint arXiv:1312.3005*, 2013.
- [67] F. Huang, X. Zhang, Z. Zhao, J. Xu, and Z. Li, “Image–text sentiment analysis via deep multimodal attentive fusion,” *Knowledge-Based Systems*, vol. 167, pp. 26–37, 2019.
- [68] A. Zadeh, M. Chen, S. Poria, E. Cambria, and L.-P. Morency, “Tensor fusion network for multimodal sentiment analysis,” *arXiv preprint arXiv:1707.07250*, 2017.
- [69] A. Zadeh, P. P. Liang, N. Mazumder, S. Poria, E. Cambria, and L.-P. Morency, “Memory fusion network for multi-view sequential learning,” in *Thirty-Second AAAI Conference on Artificial Intelligence*, 2018.
- [70] G. Cai and B. Xia, “Convolutional neural networks for multimedia sentiment analysis,” in *Natural Language Processing and Chinese Computing*, pp. 159–167, Springer, 2015.
- [71] Z. Zhao, H. Zhu, Z. Xue, Z. Liu, J. Tian, M. C. H. Chua, and M. Liu, “An image-text consistency driven multimodal sentiment analysis approach for social media,” *Information Processing & Management*, vol. 56, no. 6, p. 102097, 2019.
- [72] M. Yazdani and L. Manovich, “Predicting social trends from non-photographic images on twitter,” in *2015 IEEE international conference on big data (big data)*, pp. 1653–1660, IEEE, 2015.
- [73] H. Zhang and J. Pan, “Casm: A deep-learning approach for identifying collective action events with text and image data from social media,” *Sociological Methodology*, vol. 49, no. 1, pp. 1–57, 2019.
- [74] O. Ajao, D. Bhowmik, and S. Zargari, “Fake news identification on twitter with hybrid cnn and rnn models,” in *Proceedings of the 9th international conference on social media and society*, pp. 226–230, 2018.
- [75] H. Mouzannar, Y. Rizk, and M. Awad, “Damage identification in social media posts using multimodal deep learning,” in *ISCRAM*, 2018.
- [76] X. He and L. Deng, “Deep Learning for Image-to-Text Generation: A Technical Overview,” *IEEE Signal Processing Magazine*, vol. 34, no. 6, pp. 109–116, 2017.
- [77] C. Wang, H. Yang, C. Bartz, and C. Meinel, “Image captioning with deep bidirectional LSTMs,” in *Proceedings of the 24th ACM international conference on Multimedia*, pp. 988–997, 2016.
- [78] X. Wang, Y. Peng, L. Lu, Z. Lu, and R. M. Summers, “TieNet: Text-Image Embedding Network for Common Thorax Disease Classification and Reporting in Chest X-Rays,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2018.

- [79] J. Yu, J. Li, Z. Yu, and Q. Huang, “Multimodal Transformer with Multi-View Visual Representation for Image Captioning,” *IEEE Transactions on Circuits and Systems for Video Technology*, pp. 1–1, 2019.
- [80] J. Pavlopoulos, V. Kougia, and I. Androutsopoulos, “A Survey on Biomedical Image Captioning,” in *Proceedings of the Second Workshop on Shortcomings in Vision and Language*, pp. 26–36, 2019.
- [81] M. Chohan, A. Khan, M. S. Mahar, S. Hassan, A. Ghafoor, and M. Khan, “Image Captioning using Deep Learning: A Systematic,” *Image*, vol. 11, no. 5, 2020.
- [82] “Load images | tensorflow core.” https://www.tensorflow.org/tutorials/load_data/images?hl=en. (Accessed on 08/10/2020).
- [83] J. Camacho-Collados and M. T. Pilehvar, “On the role of text preprocessing in neural network architectures: An evaluation study on text categorization and sentiment analysis,” *arXiv preprint arXiv:1707.01780*, 2017.
- [84] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, *et al.*, “Huggingface’s transformers: State-of-the-art natural language processing,” *ArXiv*, pp. arXiv–1910, 2019.
- [85] “Transfer learning and fine-tuning | tensorflow core.” https://www.tensorflow.org/tutorials/images/transfer_learning?hl=en. (Accessed on 08/11/2020).
- [86] “Custom models: Mixup transformers with tf.keras code · issue #1350 · huggingface/transformers.” <https://github.com/huggingface/transformers/issues/1350>. (Accessed on 08/11/2020).
- [87] “Roberta — transformers 3.0.2 documentation.” https://huggingface.co/transformers/model_doc/roberta.html. (Accessed on 08/11/2020).
- [88] “Classification on imbalanced data | tensorflow core.” https://www.tensorflow.org/tutorials/structured_data/imbalanced_data?hl=en#optional_set_the_correct_initial_bias. (Accessed on 08/11/2020).
- [89] A. Karpathy, “A recipe for training neural networks.” <http://karpathy.github.io/2019/04/25/recipe/#2-set-up-the-end-to-end-trainingevaluation-skeleton--get-dumb-baseline> (Accessed on 08/11/2020).
- [90] “Transfer learning & fine-tuning | tensorflow core.” https://www.tensorflow.org/guide/keras/transfer_learning. (Accessed on 08/11/2020).

Appendix A

Model Architecture

This appendix includes:

- Outline architecture diagram of the model: Fig. A.1
- Close-up on the auxiliary text features submodel with layer input and output shapes: Fig. A.2
- Close-up on the input submodels concatenation and the output with layer input and output shapes: Fig. A.3

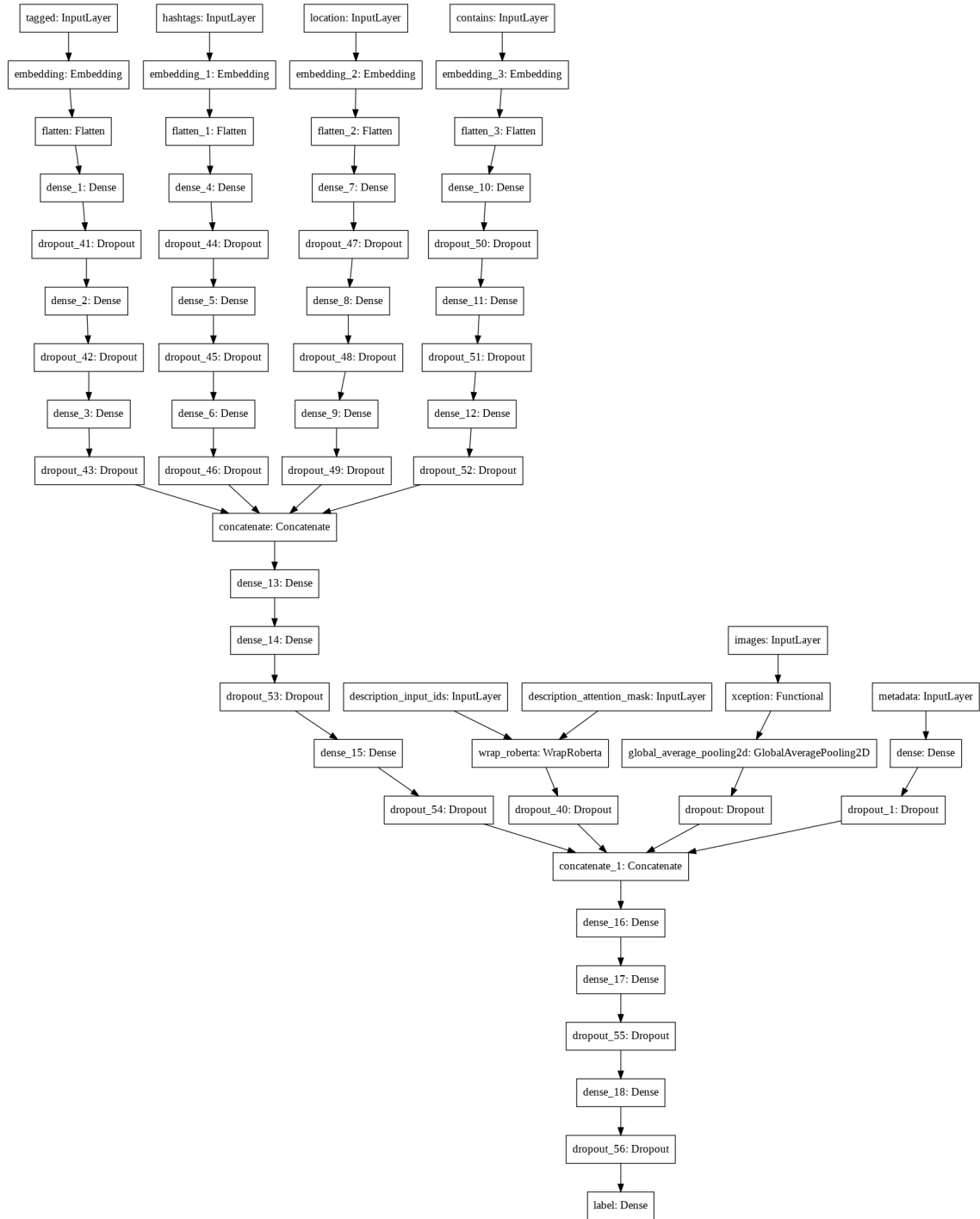


Figure A.1: Architecture diagram of the model: full model outline

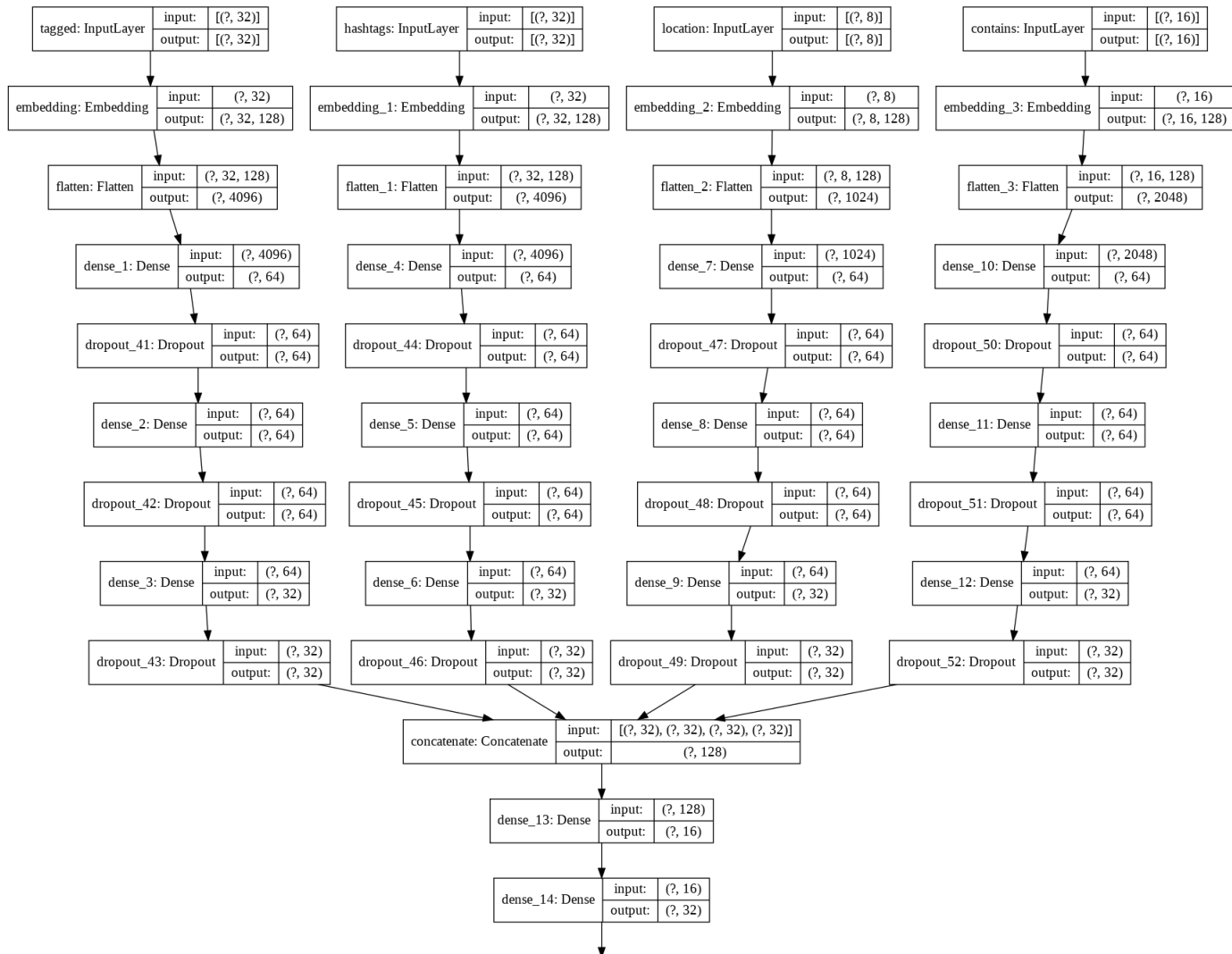


Figure A.2: Architecture diagram of the model: part 1

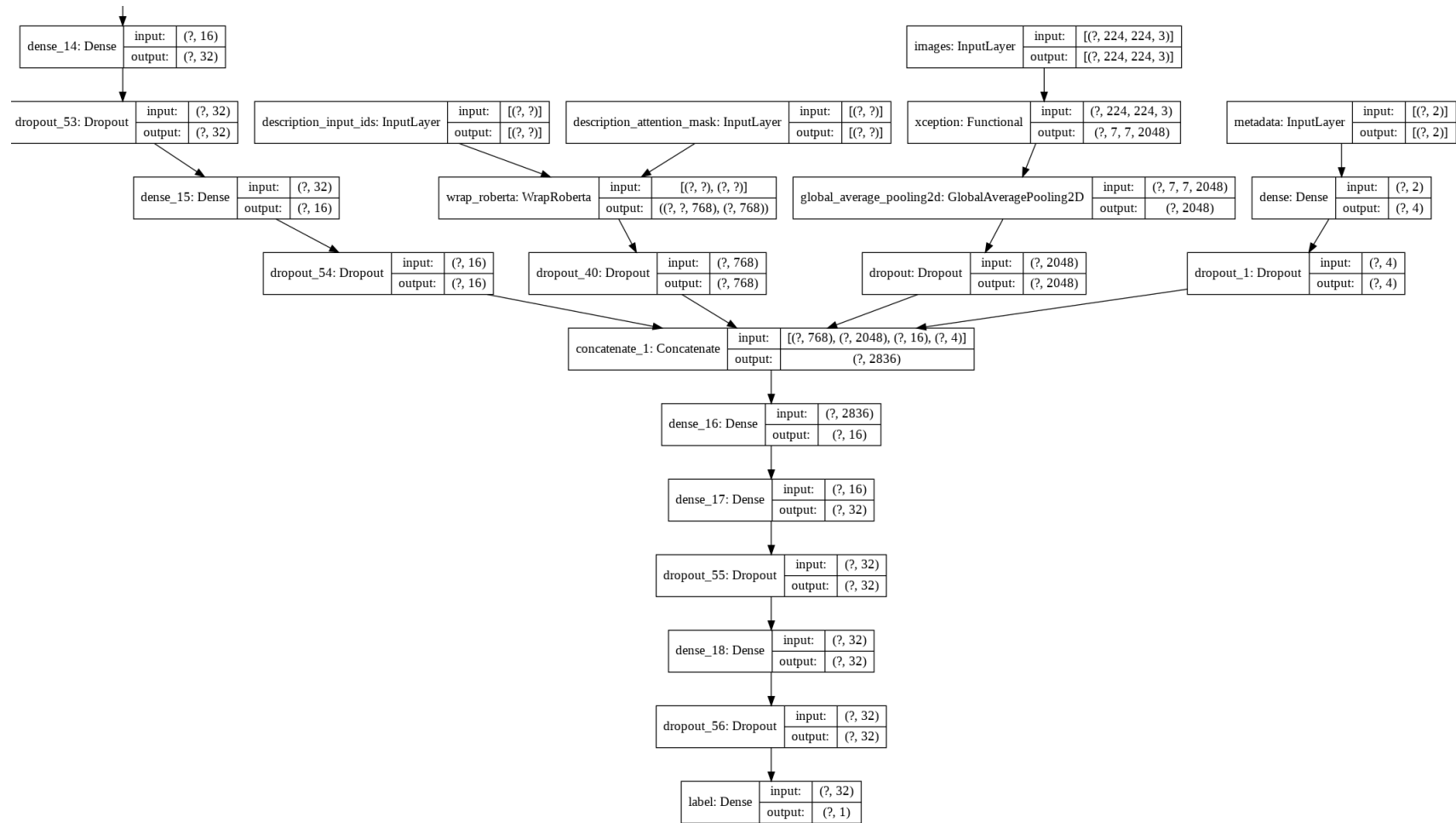


Figure A.3: Architecture diagram of the model: part 2

Appendix B

Instagram Scraper - Google Colab Setup

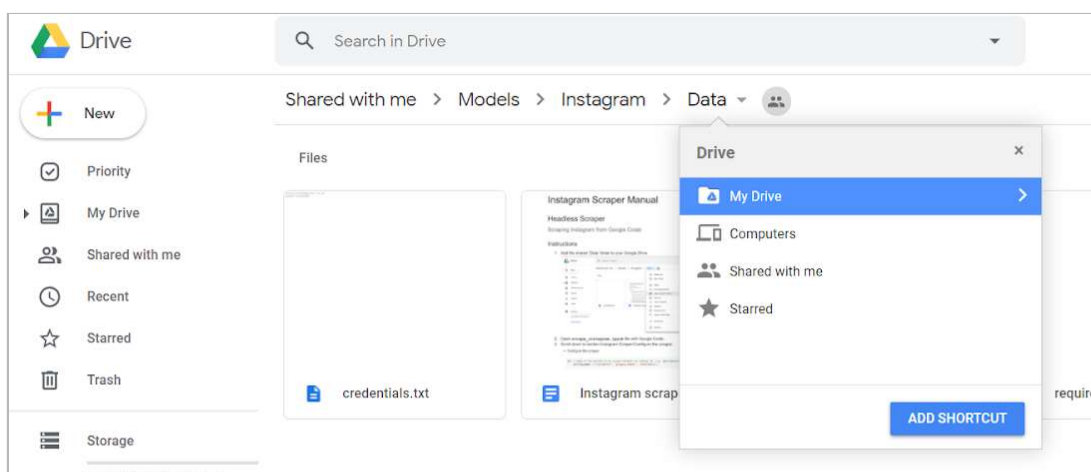
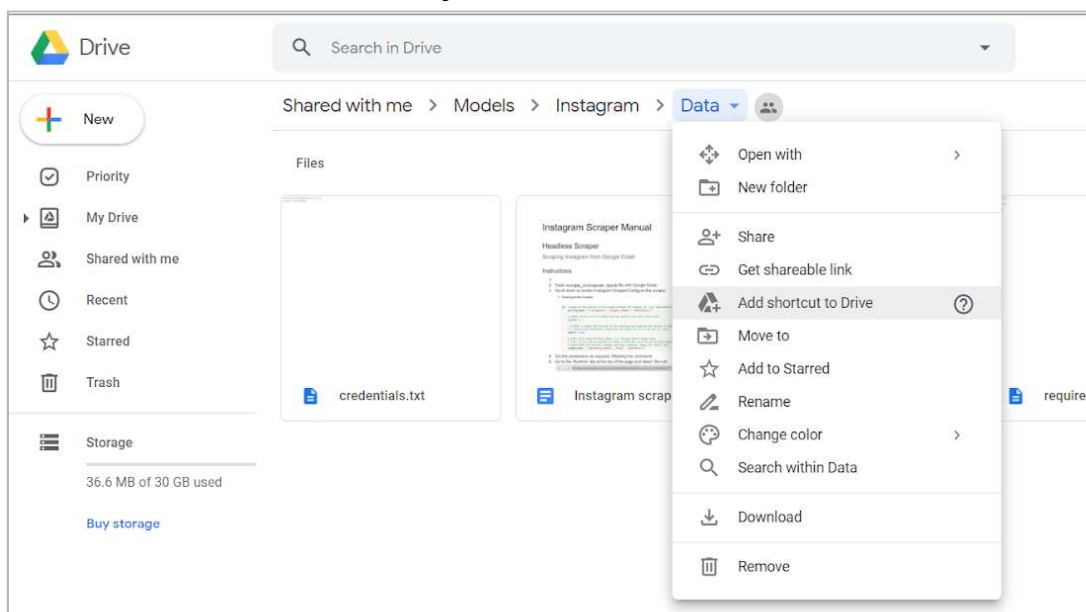
This appendix includes:

- Instagram Scraper manual: section B.1
- Google Colab setup for the Instagram Scraper: section B

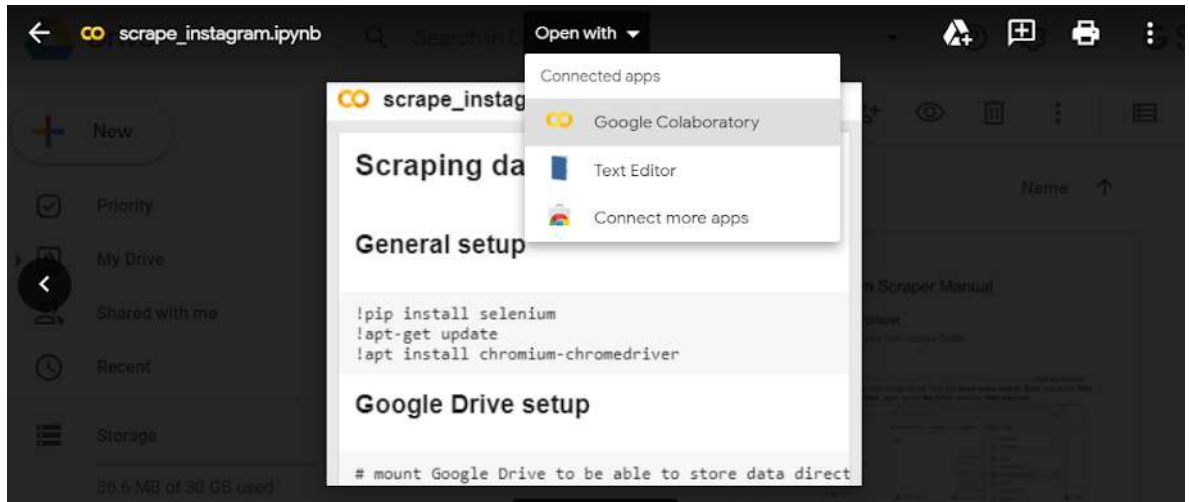
B.1 Instagram Scraper Manual

Instructions

1. (Only if you are using the scraper for the first time from your account) Add the shared 'Data' folder to your Google Drive. Click the **down arrow next to 'Data'** and select **'Add shortcut to Drive'**. Next, select **'My Drive'** and click **'Add shortcut'**.



2. Open `scrape_instagram.ipynb` file with Google Colaboratory.



3. At the top of the page, you will find a '**Configure the scraper**' section. Set the parameters as required, following the comments.
For target types and Instagram profile names, please remember to put quotation marks around each entry and separate consecutive entries with commas. Please also pay attention to the square brackets (make sure you don't delete them).

▼ Scraping data from Instagram

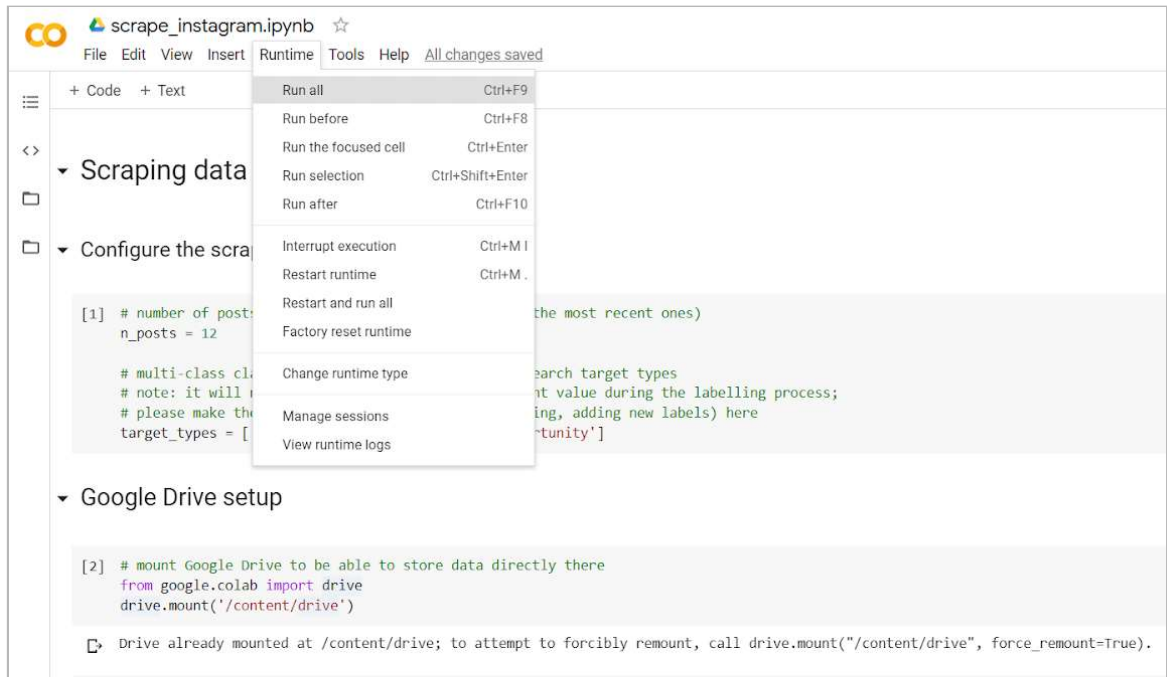
▼ Configure the scraper

```
# number of posts to be scraped from each profile (the most recent ones)
n_posts = 10

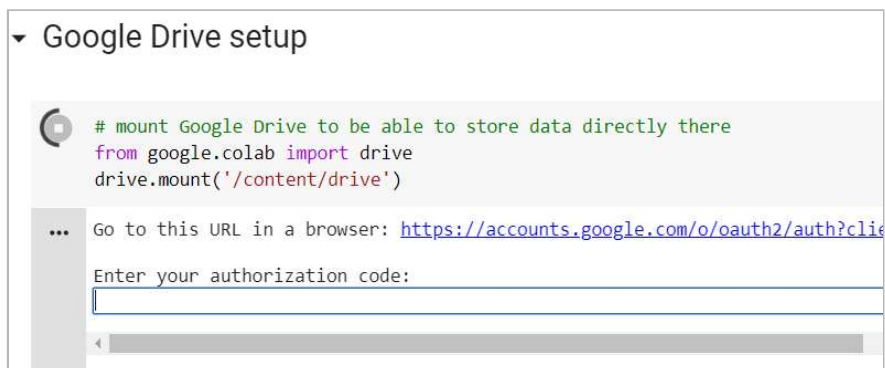
# multi-class classification labels, i.e. business search target types
# note: it will not be possible to choose a different value during the labelling process;
# please make the necessary changes (editing, removing, adding new labels) here
target_types = ['investment', 'financial', 'corporate_change', 'jobs', 'business_development', 'techno

# names of the profiles to be scraped (without the leading '@', e.g. '@teslamotors' -> 'teslamotors')
# (remove those which have already been scraped, add new ones as required)
profile_names = ['teslamotors', 'glasgow_chamber', 'ukhmtreasury']
```

4. Go to the 'Runtime' tab at the top of the page and select 'Run all'.



5. In the 'Google Drive setup' section (just below the 'Configure the scraper' section), follow the displayed instructions to authorise Colab with Drive.

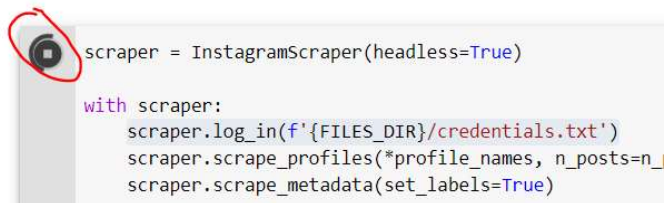


6. Scroll down to the bottom of the page. The algorithm will first scrape the main pages of the selected profiles (informing of that in the displayed log messages). Next, it will display, one by one, a screenshot of a post, with a link you can click if you want to see the full post in a new tab.
7. Below, there will be a line asking for a class label for the post. **Please enter 1 if the post belongs to the positive class (i.e. is of interest to businesses, is a search target) or 0 if it doesn't.** Next, press 'Enter'.
8. If you entered 1 ('positive' class), the algorithm will ask for a multi-class label. **Please choose one or more numbers** corresponding to one of the displayed labels (see the image below) and press 'Enter'.

See the full website here: <https://www.instagram.com/p/CBBKu2hFnPP/>
Please enter label code (0 for negative, 1 for positive):
1
Please enter label code (0 for marketing_content, 1 for trend, 2 for opportunity) - or multiple labels separated by commas:
0, 2

9. The algorithm will terminate after all the posts data is scraped. **Alternatively, you can terminate it by clicking the 'stop' icon** at the top left corner of the code cell:

▼ Launch the scraper



10. The results will be stored in a `scraped_data` folder created in the Drive.

B.2 Instagram Scraper - Google Colab Setup

▼ Scraping data from Instagram

▼ Configure the scraper

```
# number of posts to be scraped from each profile (the most recent <n_posts> ones,
# skipping the first <skip_initial> posts)
n_posts = 2
skip_initial = 0

# multi-class classification labels, i.e. business search target types
# note: it will not be possible to choose a different value during the labelling process
# please make the necessary changes (editing, removing, adding new labels) here
target_types = ['investment', 'financial', 'corporate_change', 'jobs',
                'business_development', 'technology', 'start_up', 'funding', 'new_product']

# names of the profiles to be scraped (without the leading '@',
# e.g. '@teslamotors' -> 'teslamotors')
profile_names = ['teslamotors', 'bbcnews', 'ukhmtreasury']
```

▼ Google Drive setup

```
# mount Google Drive to be able to store data directly there
from google.colab import drive
drive.mount('/content/drive')
```

 Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount('/content/drive').

```
FILES_DIR = '/content/drive/My Drive/Data'
OUTPUT_DIR = f'{FILES_DIR}/scraped_data'
```

▼ Dependencies installation

```
!pip install selenium
!apt-get update
!apt install chromium-chromedriver
```

▼ Instagram Scraper

▼ Import InstagramScraper

```
!cp '/content/drive/My Drive/Data/miniscraper.py' .
```

```
from miniscraper import InstagramScraper
```

▼ Launch the scraper

```
scraper = InstagramScraper(headless=True)

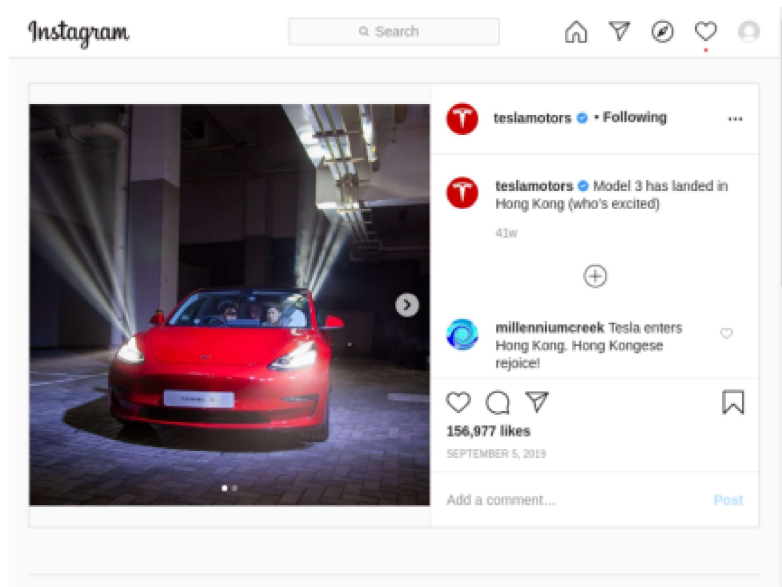
with scraper:
    scraper.log_in(f'{FILES_DIR}/credentials.txt', wait_time=3)
    scraper.scrape_profiles(*profile_names, n_posts=n_posts, skip_initial=skip_initial,
    scraper.scrape_metadata(set_labels=True, multiclass_labels=target_types)
    scraper.log_out()

scraper.download_images(data_dir=OUTPUT_DIR)
scraper.store_data(file_name=f'{OUTPUT_DIR}/instagram_dataset.csv', append=True)
```



DEBUG:miniscraper:Initialising a browser driver
DEBUG:miniscraper:Reading user credentials from /content/drive/My Drive/Data/credenti
INFO:miniscraper:Logging in as dmtargs

DEBUG:miniscraper:Logged in
INFO:miniscraper:Scraping 3 profile(s)
INFO:miniscraper:Scraping 'HM Treasury' profile (@ukhmtreasury)
DEBUG:miniscraper:Completed
INFO:miniscraper:Scraping verified 'BBC News' profile (@bbcnews)
DEBUG:miniscraper:Completed
INFO:miniscraper:Scraping verified 'Tesla' profile (@teslamotors)
DEBUG:miniscraper:Completed
INFO:miniscraper:Scraping metadata for 6 posts

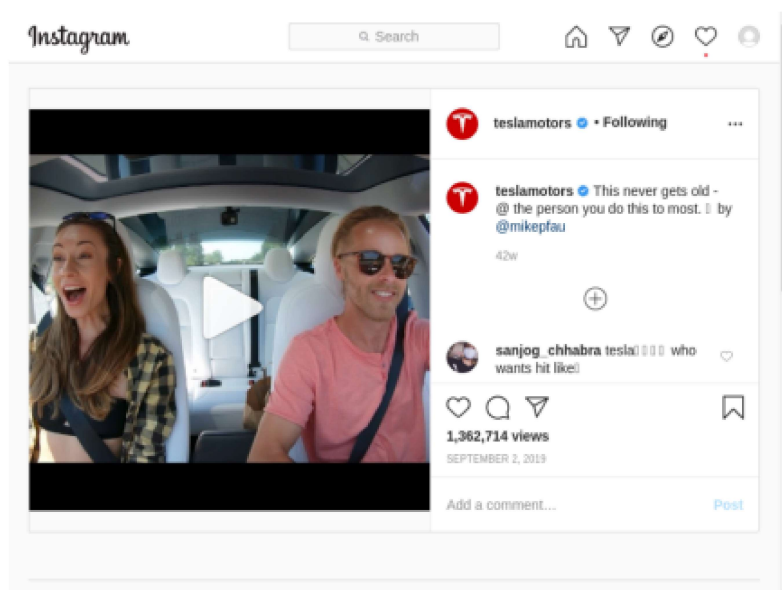


See the full website here: <https://www.instagram.com/p/B2CoQR7BtWL/>
Please enter label code (0 for negative, 1 for positive):

1

Please enter label code (0 for investment, 1 for financial, 2 for corporate_change, 3 for other):
0

Assigned labels: investment



See the full website here: <https://www.instagram.com/p/B1660kVBPJR/>
Please enter label code (0 for negative, 1 for positive):

Appendix C

The Dataset

Table C.1: Example dataset entries (part 1)

	binary_label	multiclass_label	user_name	profile_name	verified_profile	post_url
index						
1503	1	new_product	childsfarm	Childs Farm	True	https://www.instagram.com/p/CCi6XAMI1Kc/
403	0	NaN	paintersonline	PaintersOnline	False	https://www.instagram.com/p/CCiVrpaDE7I/
1950	1	technology;new_product	nasa	NASA	True	https://www.instagram.com/p/CBgoTijSsz/
1231	1	jobs;business_development	business.standard	Business Standard	True	https://www.instagram.com/p/CCikTgmH3jD/
1079	1	business_development	wsj	The Wall Street Journal	True	https://www.instagram.com/p/CCJZqUhgKfC/

Table C.2: Example data entries (part 2)

	date_time	description	alt_text	likes	img_class
index					
1503	2020-07-13T18:30:10.000Z	Save your pumps! Remember, our 500ml bottles will start to look like this soon, with an easy to pour lid rather than a pump! So if you love your pumps, and are coming to the end of an existing bottle and want to keep them to reuse in the future, please save yours now!If you are swapping strai...	Photo by Childs Farm on July 13, 2020. Image may contain: 1 person	260	FFVAD
403	2020-07-12T09:11:54.000Z	Have you seen David Bellamy's top tips for watercolour painting yet? https://bit.ly/3fgzM7V Must-see advice for all watercolourists!	Photo by PaintersOnline on July 12, 2020. Image may contain: 1 person, sitting, screen and indoor	25	FFVAD
1950	2020-06-16T20:47:24.000Z	Meet the people behind our next Mars rover It takes thousands of people and years of hard work to get a spacecraft from Earth to Mars. So when our Perseverance rover touches down on the Martian surface, it will be because of the talented minds that helped to make it happen. The team is on tra...	It takes thousands of people and years of hard work to get a spacecraft from Earth to Mars. So when our Perseverance rover touches down on the Martian surface, it will be because of the talented minds that helped to make it happen. The team is on track to launch Perseverance on July 20 and land...	2085374	FFVAD
1231	2020-07-16T17:51:16.000Z	Education technology start-ups are aggressively ramping up their headcount to cater to the new demands as the Covid-19 situation adds new tailwinds to the sector. Most players are expecting their headcount to double by the end of the year as they seek to fill positions across sales, technology, ...	Photo by Business Standard on July 16, 2020. Image may contain: 1 person, text	723	FFVAD
1079	2020-07-02T16:45:39.000Z	There is widespread scientific and medical consensus that face masks are a key part of the public policy response for tackling the coronavirus pandemic. So why are they still controversial? Although many European countries and U.S. states have made masks mandatory in shops or on public transport...	There is widespread scientific and medical consensus that face masks are a key part of the public policy response for tackling the coronavirus pandemic. So why are they still controversial? Although many European countries and U.S. states have made masks mandatory in shops or on public transport...	20879	FFVAD

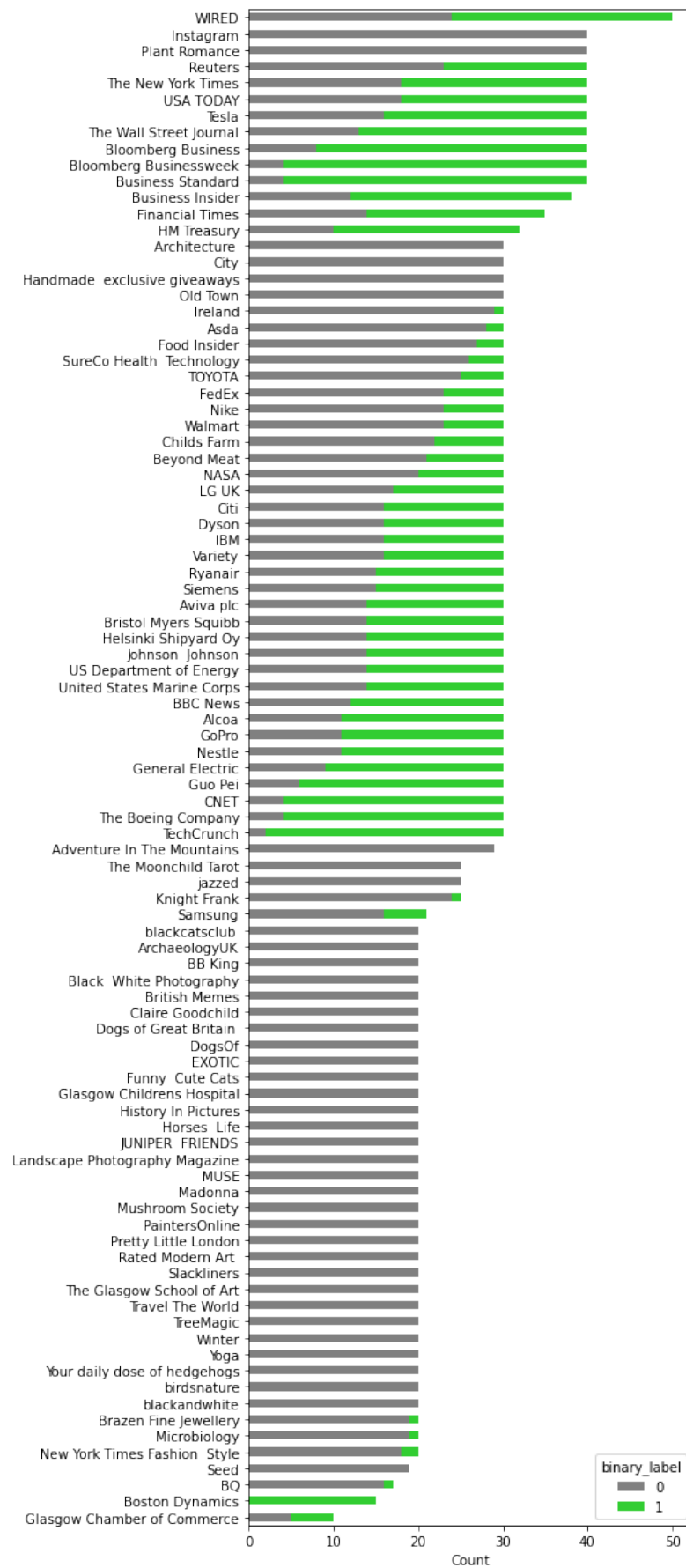


Figure C.1: Positive and negative examples per profile

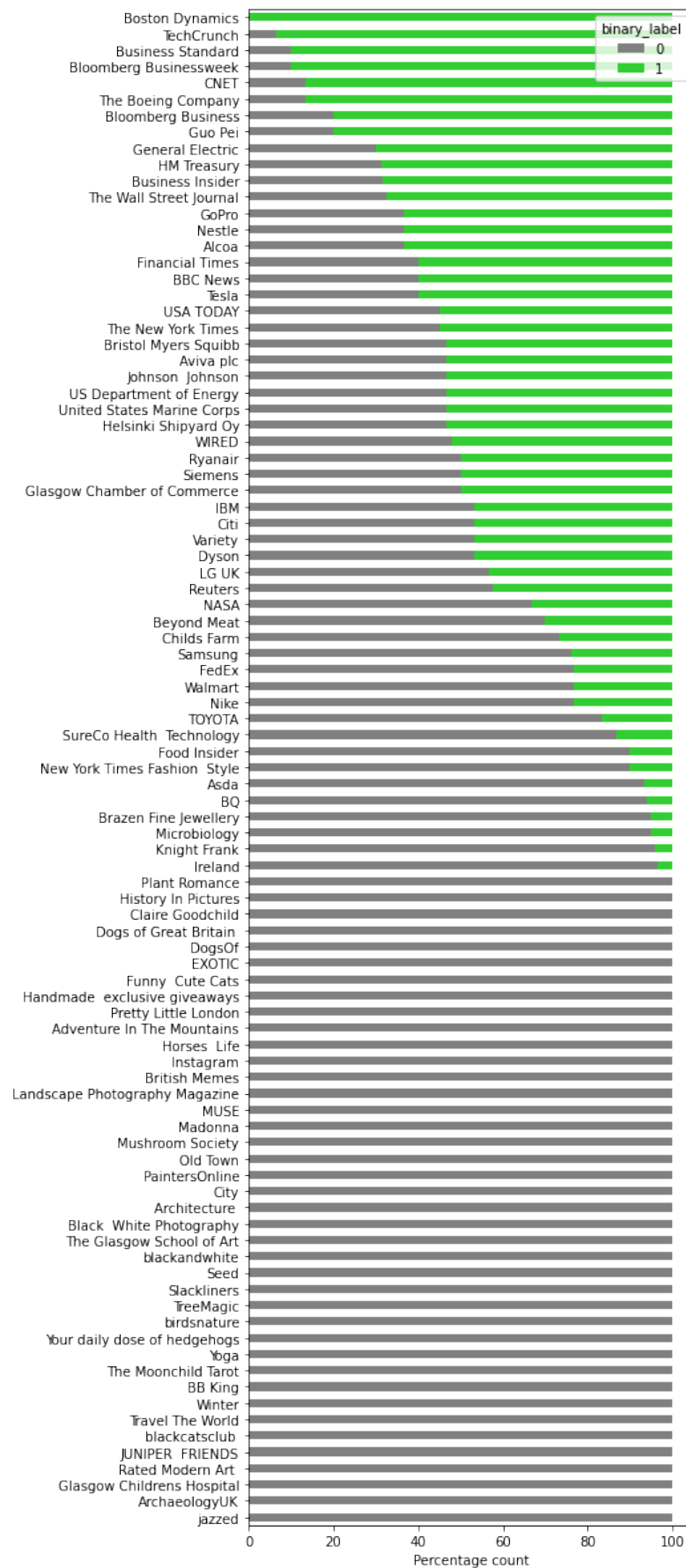


Figure C.2: Percentages of positive and negative examples per profile

Appendix D

Results

This appendix includes:

- Enlarged plots of the learning curves: Fig. D.1
- Enlarged images for the misclassified examples from the test set: Fig. D.2
- Table with post descriptions and alternative texts for the misclassified examples from the test set: Table D.1

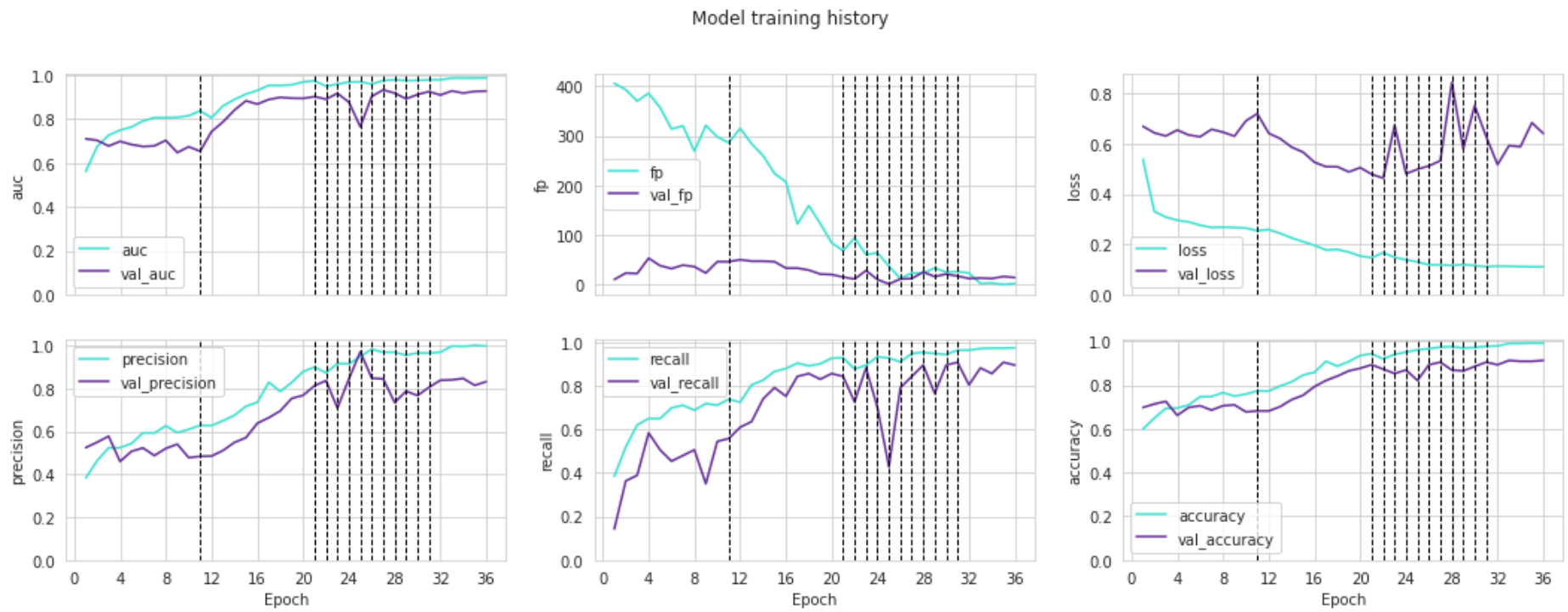


Figure D.1: Model learning curves for the training and validation sets



Figure D.2: Misclassified examples from the test set

Table D.1: Misclassified examples from the test set
 (‘true’ - true label, ‘proba’ - predicted probability, ‘pred’ - predicted label)

index	true	proba	pred	profile_name	description	alt_text
1843	1	0.308415	0	Johnson & Johnson	It hasnt always been easy to show my authentic self to the people around me. When I started working in the corporate world at 21, I was lost, depr...	Photo by Johnson & Johnson on October 11, 2018. Image may contain: 1 person
1978	1	0.308415	0	Alcoa	Alcoa Australia has presented artist @april_pine with a \$10,000 Alcoa Aluminium Award for her sculpture, Flutter, which is on show now @Sculptureb...	Alcoa Australia has presented artist @april_pine with a \$10,000 Alcoa Aluminium Award for her sculpture, Flutter, which is on show now @Sculptureb...
1645	1	0.308415	0	IBM	We're taking you inside our quantum stack to show you how it works. Explore by clicking on the link in our bio. #Think2020	Photo by IBM on May 07, 2020.
1836	1	0.491330	0	Johnson & Johnson	Did you know that in 2016 there were 1.8 million new HIV infections worldwide? And 43% of those infections were in eastern and southern Africa, wi...	Did you know that in 2016 there were 1.8 million new HIV infections worldwide? And 43% of those infections were in eastern and southern Africa, wi...
1423	0	0.510702	1	United States Marine Corps	Stepping On The Beach Marines with the @31stmeu carry a Combat Rubber Raiding Craft (CRCC) during a squad competition at Kin Blue, Okinawa, Japan...	Stepping On The Beach Marines with the @31stmeu carry a Combat Rubber Raiding Craft (CRCC) during a squad competition at Kin Blue, Okinawa, Japan...
22	0	0.851358	1	Bloomberg Business	John Boltons pending tell-all is living up to its publishers hype as the book President Donald Trump doesnt want you to read." . Excerpts from the...	Photo by Bloomberg Business on June 19, 2020. Image may contain: 2 people, text
1073	0	0.990960	1	The Wall Street Journal	In Japans host clubs, women come to be entertained with Champagne and sweet talk by teams of well-groomed young male staff. But now the clubsan of...	In Japans host clubs, women come to be entertained with Champagne and sweet talk by teams of well-groomed young male staff. But now the clubsan of...
2125	0	0.999184	1	Aviva plc	At Aviva we think ahead to keep our employees safe. Weve been doing it for decades. Especially at another time of great uncertainty World War II....	At Aviva we think ahead to keep our employees safe. Weve been doing it for decades. Especially at another time of great uncertainty World War II....
1857	0	0.999701	1	Johnson & Johnson	Theres teamwork, and then theres lacing up your running shoes and sprinting through the rocky terrain of Sweden before diving into the Baltic Sea ...	Theres teamwork, and then theres lacing up your running shoes and sprinting through the rocky terrain of Sweden before diving into the Baltic Sea ...
1772	0	0.999748	1	GoPro	#F1 is back. Flashback to last years festivities with #GoProAthlete @danielricciardo, #GoProMAX, + #GoProHERO8 Black. Wish Daniel a successful ...	#F1 is back. Flashback to last years festivities with #GoProAthlete @danielricciardo, #GoProMAX, + #GoProHERO8 Black. Wish Daniel a successful ...
34	0	0.999989	1	The Wall Street Journal	Three months after Perus president deployed soldiers to the streets in implementing one of the Western Hemispheres strictest coronavirus lockdowns...	Photo by The Wall Street Journal on June 18, 2020. Image may contain: 1 person, standing
1075	0	0.999997	1	The Wall Street Journal	Coronavirus has taken a staggering toll on Europes oldest citizens that is shaping public-health policy as authorities look ahead to the fall and ...	Coronavirus has taken a staggering toll on Europes oldest citizens that is shaping public-health policy as authorities look ahead to the fall and ...

Appendix E

Instarget User Manual

User manual for Instarget - a Python-based system for identification of business signals from Instagram posts.

The following functionality types are implemented in the package:

- a framework for collection and annotation of Instagram data (scraper module)
- preprocessing of the collected data, in particular - textual data, and preparing it for deep learning model training (preprocessing module)
- definition of a multi-modal, multi-input deep learning model, including pretrained models for text and image processing (models module)
- construction of a full deep transfer learning pipeline (pipeline module)
- quantitative and qualitative assessment of the model performance (evaluation.py script in the pipeline module).

E.1 Installation and usage

The back-end of the system was constructed based on a set of Python scripts, organised in several modules. Some functionalities were implemented in Google Colab notebooks (.ipynb) format.

E.1.1 Local installation

The usage of the system on a local machine requires Python 3.8 or newer to be installed. Please refer to python.org website for installation or update details.

In order to use the system on a local machine, please follow these steps:

1. Navigate into folder instarget (which holds the instarget package):

```
cd <path-to-the-main-folder>
```

2. Install the package:

```
python setup.py install
```

(adjusting the command to the operating system, e.g. swapping python for python3 if needed). This will also install all the package requirements listed in the requirements.txt file.

The python scripts in the package contain usage examples. They can be executed using the following command:

```
python <path/to/the/script>/name_of_the_script.py
```

Please note that in some cases, additional files - e.g. data files which could not have been made available with this project - might be needed to present the full functionality. However, data can be easily collected using the `miniscrapper.py` script in the `scraper` module. Please refer to the `README.md` file of the `scraper` module.

E.1.2 Using Instarget on Google Colab

Some of the main functionalities of the system, including construction of the pipeline as well as training of the model, were created in the form of Google Colab notebooks.

In order to use Instarget in Google Colab, please follow these instructions:

1. Upload the package to Google Drive.
2. Use `scrape_instagram.ipynb` notebook (please refer to the `scraper` module `README.md` file) to collect data. Alternatively, collect the data locally using the `miniscrapper.py` script and upload it to a location of choice in the Google Drive.
3. Open `colab_setup.ipynb` notebook in Google Colab. Edit the paths to point to the package and data directory, ignoring the git setup.
4. Open any of the notebooks to be executed, e.g. `full_pipeline.ipynb` in `pipeline` module. Edit the path to the `colab_setup.ipynb` file to point to the location of the notebook in the Drive.
5. Delete or comment-out line saying `UPDATE_GIT` in the first cell of the notebook. This was originally set to `True` to work with the package associated with a GitLab repository.
6. For `full_pipeline.ipynb` or `bert_pipeline.ipynb`, it is recommended to choose GPU as the hardware accelerator (the option can be accessed in the *Runtime* tab in the menu under *Change runtime type*).
7. In the Runtime tab, select *Run all* to execute all cells of the notebook.